CUADERNO DE ESTUDIO · DOCUMENTACIÓN TÉCNICA

AVENTURA 2D

La granja de La Tranquera



CÓMO LO PROGRAMÉ, PASO A PASO

Godot · GDScript · radio en vivo · guardado automático
exportación a PC, Android, web e iPhone

TRABAJO PRÁCTICO · 2026
CON GUION PARA EL ORAL, 30 PREGUNTAS Y RESPUESTAS,
Y EL CÓDIGO EXPLICADO LÍNEA POR LÍNEA

Índice

Prólogo: cómo usar este cuaderno pág. 3

1. La idea y el plan 5

2. Godot y la estructura del proyecto 8

3. El personaje y los controles 12

4. El menú, los ajustes y la intro 16

5. La granja: cultivos, crecimiento y cosecha 23

6. Economía, tienda y empleados 27

7. El mundo: mapa, clima y vida 32

8. El guardado: que no se pierda nada 36

9. La radio: el problema más difícil 40

10. Llevarlo a todos lados: exportar y publicar 48

11. Errores que me encontré y cómo los resolví 54

12. Guion para el oral y preguntas de defensa 56

Anexo A. Glosario técnico 61

Anexo B. Chuleta de una página 62

Anexo C. Código completo de referencia 63

CÓMO ESTÁ ARMADO CADA CAPÍTULO

1. **El relato:** qué quería lograr y por qué tomé cada decisión (está escrito en primera persona, como si te lo contara a vos).
2. **El código:** los fragmentos reales del proyecto, con el nombre del archivo y las líneas exactas.
3. **Las capturas:** cómo se ve eso en el juego terminado.
4. **Para decir en el oral:** un párrafo corto, listo para leer o memorizar.
5. **Preguntas y respuestas:** lo que te pueden preguntar y cómo contestarlo sin trabarte.

Prólogo: cómo usar este cuaderno

Este cuaderno cuenta cómo hice **Aventura 2D**, un juego de granja en pixel art con radio en vivo, guardado automático y versiones para computadora, celular Android, navegador e iPhone. No es un manual de Godot ni una lista de funciones: es el **recorrido real** de cómo pasé de una idea a un juego que se puede descargar y jugar.

Lo escribí en primera persona porque la idea es que puedas **defenderlo**: si te preguntan "¿por qué usaste esto y no aquello?", la respuesta no está en un tutorial genérico, está en las decisiones que tomé yo, con los problemas que aparecieron en el camino.

Qué vas a encontrar

- **El porqué antes del cómo.** Cada sistema empieza explicando qué problema resolvía. Después viene el código. Si entendés el problema, el código se lee solo.
- **Código real.** Todos los fragmentos salen del proyecto: `player.gd`, `world.gd`, `game_save.gd`, `start_menu.gd`, `intro.gd`, `scene_transition.gd` y el workflow de GitHub Actions.
- **Capturas del juego funcionando** y del editor de Godot, para que puedas mostrar de dónde sale cada cosa.
- **Guiones para el oral** y **preguntas con respuesta**. Al final hay un guion de cinco minutos, treinta preguntas frecuentes y una chuleta de una página.

Cómo estudiarlo (mi recomendación)

1. **Primera pasada (40 minutos):** leé solo los relatos y mirá las capturas. Quedate con la historia: idea, problemas, soluciones.
2. **Segunda pasada (2 horas):** volvé capítulo por capítulo y leé el código. No lo memorices: entendé qué entra y qué sale de cada función.
3. **Tercera pasada (1 hora):** tapá las respuestas del capítulo 12 y contestá en voz alta. Si te trabás, volvé al capítulo.
4. **El día anterior:** leé solo el guion del oral y la chuleta del Anexo B.

DATOS DEL PROYECTO, PARA CITAR DE MEMORIA

Dato	Valor
Motor	Godot 4.7.2 (GDScript, renderer GL Compatibility)
Resolución interna	320 × 180 (pixel art), escalada a 960 × 540 y a pantalla completa
Escenas principales	`start_menu.tscn`, `intro.tscn`, `main.tscn`
Script central	`world.gd` — 1.904 líneas
Scripts totales	7 propios (+2 de ejemplo del addon)
Mapa	5.120 × 5.120 píxeles de mundo
Recursos gráficos	292 imágenes PNG del pack *Super Retro World — Farming*
Radios	6 emisoras argentinas (5 por streaming + LU6 Mar del Plata)
Autoguardado	Cada 20 segundos y al cerrar el juego
Plataformas publicadas	Windows (ZIP), Android (APK), Web (GitHub Pages), iOS (.ipa firmada)

> *"Un juego no es una idea: es una idea más cuatrocientas decisiones chiquitas. Este cuaderno es el registro de esas decisiones."*

Convenciones del texto

- Los recuadros **verdes** son frases que podés decir tal cual en el oral.
- Los recuadros **amarillos** son datos o advertencias que conviene recordar.
- Los recuadros **rojos** son errores reales que tuve y cómo los resolví (sirven muchísimo en la defensa: muestran criterio).
- Los bloques negros son código del proyecto. El nombre del archivo y las líneas están en la barra de arriba.

1 La idea y el plan

Empecé con una imagen muy simple: alguien que deja la ciudad, hereda una granja chica y empieza de cero. No quería un juego de terror ni de plataformas: quería algo **tranquilo**, con progreso visible, donde cada día se vea un poquito mejor que el anterior. Esa es la razón de fondo por la que elegí el género *farming sim*.

La historia la terminé escribiendo en cinco pantallas de introducción. El protagonista se llama Mateo: diez años sellando papeles en una oficina, un día hereda una granja junto a un pueblo y se va. El juego arranca justo cuando llega.

> *"Mateo llevaba diez años trabajando para el municipio. Sellaba papeles, cumplía horarios y cruzaba avenidas llenas de bocinas. Al terminar el día, el ruido seguía con él."*
> *— Primera pantalla de la intro (`intro.gd`, línea 17)*

1.1 Las decisiones que tomé antes de escribir una línea

Antes de tocar el motor hice una lista corta. Si no tomaba estas decisiones primero, después iba a estar reescribiendo todo:

Decisión	Qué elegí	Por qué
Motor	Godot 4.7.2	Gratis, liviano, exporta a PC/Android/Web/iOS desde el mismo proyecto y usa GDScript, que se parece a Python.
Vista	2D *top-down*	Es la vista clásica de los juegos de granja y no necesita modelos 3D.
Estilo	Pixel art	Permite arte chico, coherente y liviano: 292 PNG y el juego entero pesa 38 MB.
Resolución	320 × 180 interna	Es 16:9, se ve "de consola vieja" y al escalar por números enteros no deforma los píxeles.
Bucle de juego	Plantar → esperar → cosechar → vender → mejorar	Es el ciclo que hace que el jugador siempre tenga un objetivo inmediato.
Progreso	Fichas (dinero) + XP + niveles	Dos monedas: una para comprar, otra para desbloquear cosas nuevas.
Extra	Radio argentina en vivo	Le da identidad y está justificado por la historia: la vieja radio de la granja.

1.2 La resolución: por qué 320 × 180

El pixel art tiene un problema si lo escalás a cualquier tamaño: los píxeles quedan de distinto grosor y se ve sucio. La solución es definir una resolución interna chica y multiplicarla por un número entero. En `project.godot` quedó así:

project.godot · líneas 29-37 — configuración de pantalla

```
[display]

window/size/viewport_width=320
window/size/viewport_height=180
window/size/window_width_override=960
window/size/window_height_override=540
window/stretch/mode="viewport"
window/stretch/scale_mode="integer"
window/handheld/orientation=4
```

La clave está en `stretch/mode = "viewport"`: todo el juego se dibuja en 320 × 180 y después se estira. Con `scale_mode = "integer"` el estiramiento es por múltiplos exactos (×3, ×4, ×6), así que cada píxel del arte mide exactamente lo mismo en pantalla. La cámara además tiene zoom ajustable desde los ajustes (0,75× a 2×).

> POR QUÉ ESTO IMPORTA EN EL ORAL
>
> Si te preguntan "¿por qué el juego se ve pixelado?", la respuesta no es "porque es retro": es una **decisión técnica** para que el escalado sea perfecto y el juego corra en cualquier celular. Es un buen ejemplo de restricción que mejora el resultado.

1.3 El bucle de juego, escrito antes de programar

Antes de tocar código escribí el bucle en un papel. Me sirvió para saber qué datos tenía que guardar sí o sí:

1. El jugador tiene **semillas** de tres tipos: trigo, zanahoria y tomate.
2. Se acerca a un surco y presiona `E`: planta la semilla seleccionada.
3. La planta crece en tiempo real (90 s el trigo, 60 s la zanahoria, 75 s el tomate), afectada por el clima.
4. Cuando está madura, `E` la cosecha: da fichas y XP, y vuelve a empezar el ciclo.
5. Con fichas compra mejoras en la tienda: más empleados, campos grandes, suerte, velocidad.
6. Con XP sube de nivel y se desbloquean cosas (por ejemplo el granero "CERRADO · NIVEL 5").

Ese bucle define **qué hay que guardar**: posición del jugador, fichas, XP, nivel, estado de cada surco, inventario de semillas, clima actual y sus temporizadores. Si no lo hubiera escrito antes, el guardado me iba a quedar incompleto (y de hecho la primera versión lo quedó: en el capítulo 8 cuento cómo lo arreglé).

> PARA DECIR EN EL ORAL
>
> *"Antes de programar definí el bucle de juego: plantar, esperar, cosechar, vender y mejorar. Ese bucle me dijo exactamente qué variables tenía que guardar. Por eso el sistema de guardado no fue un agregado posterior: salió del diseño."*

1.4 Qué NO hice (y por qué)

También descarté cosas a propósito. Esto suele impresionar más que la lista de features:

- **No usé 3D:** el estilo no lo necesitaba y me hubiera costado el triple de trabajo y de peso.
- **No hice multijugador:** requiere servidores, cuentas y sincronización; no aportaba al objetivo.
- **No usé una base de datos:** el guardado es un archivo `ConfigFile` de 9 KB. Una base para un juego de un jugador es sobredimensionar.
- **No compré música ni voces:** el audio propio del juego es la radio; el resto del ambiente es sonido del pack.
- **No hice menús con imágenes:** los construí por código con `Control` y `StyleBoxFlat`, así no dependía de assets nuevos.

ERROR REAL #1 · QUERER ABARCAR TODO

Mi primera lista de ideas tenía: clima, animales, empleados, tienda, radio, minijuegos, misiones, viajes entre pueblos. Era imposible. La recorté a lo que se podía terminar y **dejar funcionando**. Un juego chico terminado vale más que uno enorme a medias, y en la defensa se nota.

2 Godot y la estructura del proyecto

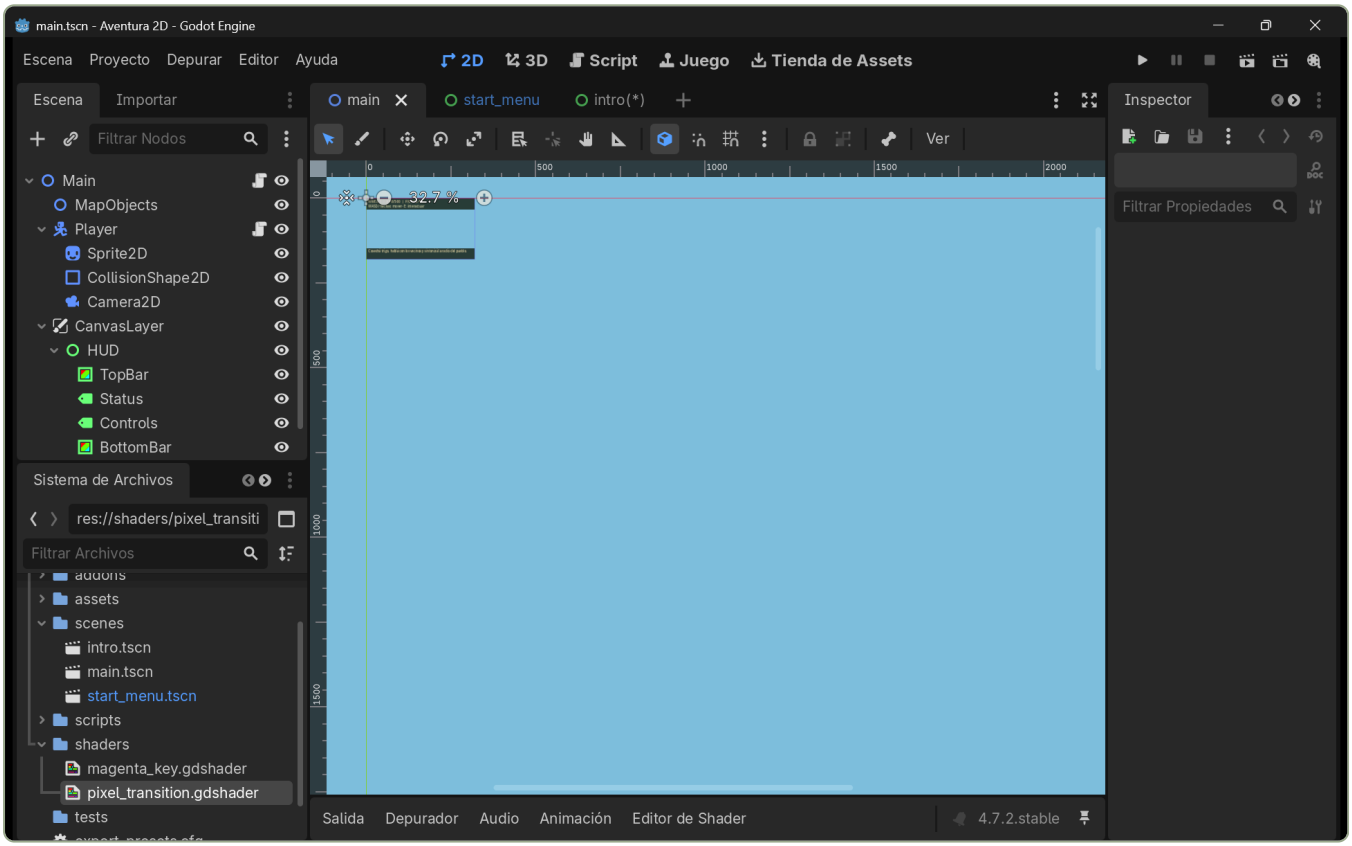
Cuando abrí Godot por primera vez la pregunta fue inevitable: *¿por dónde empiezo?* La respuesta, después de varios intentos fallidos, fue: **por el árbol de escenas**. Godot no funciona con "archivos sueltos" como otros motores: funciona con **nodos** dentro de **escenas**. Entender eso me ahorró semanas.

2.1 Qué es un nodo y qué es una escena

Un **nodo** es la unidad mínima del motor: un sprite, un sonido, una cámara, un temporizador, un cuerpo con física. Cada nodo hace una cosa. Una **escena** es un árbol de nodos guardado en un archivo `.tscn`, y ese árbol se puede instanciar dentro de otra escena como si fuera un solo nodo.

En el juego esto se ve clarísimo en la escena principal (`main.tscn`):

- `Main` (Node2D) — la raíz, con el script `world.gd`.
- `MapObjects` (Node2D) — todo lo que dibujé por código: árboles, río, tienda, radio, granero, cultivos.
- `Player` (CharacterBody2D) — el personaje, con `Sprite2D`, `CollisionShape2D` y `Camera2D`.
- `CanvasLayer/HUD` — la interfaz: barra de arriba, mensajes, controles, y todas las ventanas (tienda, radio, ruleta, pausa).



El editor con `main.tscn` abierto: a la izquierda el árbol de escenas, abajo el sistema de archivos, a la derecha el código. Así se veía el proyecto el día que terminé la radio.

PARA DECIR EN EL ORAL

“Godot trabaja con nodos: cada nodo hace una sola cosa y se combinan en un árbol. Una escena es un árbol guardado que puedo reutilizar. Por eso en el juego todo está separado: el jugador es una escena, el menú es otra, la intro es otra, y la transición entre ellas es un autoload que se mantiene vivo.”

2.2 La estructura de carpetas

Antes de programar armé las carpetas. No es un detalle estético: si el proyecto crece y todo está mezclado, después no encontrás nada. Esta fue la organización final:

Carpeta	Qué contiene	Por qué está separada
`scenes/`	`start_menu.tscn`, `intro.tscn`, `main.tscn`	Cada pantalla grande del juego es una escena independiente.
`scripts/`	7 scripts propios	La lógica va aparte del diseño de escena, así se lee mejor.
`assets/`	ícono de la app	Lo que no es del pack de arte.
`addons/SuperRetroWorld_Farming/`	883 archivos: tilesets, casas, árboles, cultivos, animales, personajes	El pack de pixel art, tal cual lo descargué.
`addons/icy-radio-streamer/`	16 archivos: la extensión nativa de radio	Es una GDExtension: código C++ compilado que expone una clase nueva a GDScript.
`shaders/`	`pixel_transition.gdshader`, `magenta_key.gdshader`	Los dos efectos que necesitaba: transición de escenas y quitar el fondo magenta de las casas.
`tests/`	pruebas de humo	Para comprobar rápido que el juego arranca después de cada cambio.
`.github/workflows/`	`build-ios.yml`	La receta para compilar el .ipa en la nube de GitHub (capítulo 10).

2.3 Los tres autoloads (y por qué son la clave)

Un **autoload** es un script que Godot carga una sola vez, al arrancar, y deja disponible en todo el juego con una ruta global (`/root/Nombre`). Es el equivalente a un “objeto global”. Usé tres, y cada uno resuelve un problema distinto:

project.godot · líneas 23-27 autoloads

```
[autoload]

GameSettings="*res://scripts/game_settings.gd"
GameSave="*res://scripts/game_save.gd"
SceneTransition="*res://scripts/scene_transition.gd"
```

Autoload	Responsabilidad	Por qué global
GameSettings	Zoom de cámara y volumen de la radio; los guarda en `settings.cfg`.	Se cambia desde el menú y se aplica en el juego: son escenas distintas.
GameSave	Leer, escribir y borrar el progreso; controla la versión del guardado.	Lo usan el menú (para saber si hay partida) y el mundo (para guardar).
SceneTransition	Cambiar de escena con el efecto de píxeles.	Si estuviera en una escena, al cambiar de escena desaparecería en medio de la animación.

Ese último punto es el que más me costó entender: **el objeto que hace la transición no puede pertenecer a la escena que está desapareciendo**. Por eso es un autoload con `process_mode = PROCESS_MODE_ALWAYS`, que sigue funcionando aunque el árbol se esté reconstruyendo.

EL ORDEN IMPORTA

Los autoloads se cargan en el orden en que aparecen. Puse **GameSettings** primero porque **GameSave** y el mundo leen ajustes al iniciar: si el orden estuviera al revés, el zoom arrancaría en 1× y se corregiría recién en el primer cambio.

2.4 Por qué construí la interfaz por código

Godot permite arrastrar botones en el editor visual. Yo hice casi toda la interfaz **por código**, con `Button.new()`, `Label.new()` y `StyleBoxFlat`. No fue capricho: tenía tres razones concretas.

1. **Consistencia:** una sola función `_make_button()` define el estilo de *todos* los botones del juego. Si cambio un color, cambia en todas las pantallas.
2. **Menos archivos:** el menú tiene tres "vistas" (inicio, ajustes, confirmación) que se muestran y ocultan. Por código es un `visible = true/false`; en el editor serían tres escenas o un lío de nodos.
3. **Escalado:** como el juego se dibuja en 320 × 180, las medidas son chiquitas (un botón de 138 × 22 píxeles). Posicionar eso a mano en el editor, con el zoom del canvas, era más lento que escribirlo.

start_menu.gd · líneas 207-231 — el estilo único de todos los botones

```
func _make_button(text: String, callback: Callable, size := Vector2(138, 25)) -> Button:
	var button := Button.new()
	button.text = text
	button.custom_minimum_size = size
	button.focus_mode = Control.FOCUS_ALL
	button.add_theme_font_size_override("font_size", 11)
	button.add_theme_color_override("font_color", TEXT_COLOR)
	button.add_theme_color_override("font_hover_color", Color.WHITE)
	button.add_theme_color_override("font_pressed_color", TEXT_COLOR)
	button.add_theme_stylebox_override("normal", _button_style(Color("435c3b")))
	button.add_theme_stylebox_override("hover", _button_style(Color("668349")))
	button.add_theme_stylebox_override("pressed", _button_style(Color("34482f")))
	button.add_theme_stylebox_override("focus", _button_style(Color("668349")))
	button.pressed.connect(callback)
	return button
```

Fijate en el detalle de `focus_mode = Control.FOCUS_ALL`: es lo que permite navegar el menú con **Tab** y **Enter**. Sin eso, el juego sería inusable en una PC sin mouse o con un joystick, y además es requisito de accesibilidad.

2.5 El renderer: por qué GL Compatibility

Godot 4 trae dos renderers: **Forward+** (más moderno, usa Vulkan) y **Compatibility** (usa OpenGL ES 3.0 / WebGL 2). Elegí Compatibility por una razón muy concreta: **la versión web**. Forward+ no corre en navegador, y yo quería que el juego se pudiera jugar desde un link sin instalar nada.

project.godot · líneas 39-44 renderer

```
[rendering]

renderer/rendering_method="gl_compatibility"
renderer/rendering_method.mobile="gl_compatibility"
textures/vram_compression/import_etc2_astc=true
environment/defaults/default_clear_color=Color(0.49, 0.76, 0.88, 1)
```

El costo: perdí efectos avanzados (sombras dinámicas, luces volumétricas, SDFGI). Como el juego es 2D pixel art, no los necesitaba. La ganancia: **un solo proyecto que exporta a cuatro plataformas** sin mantener versiones distintas. Esa decisión la volvería a tomar igual.

ERROR REAL #2 · EL COLOR DE FONDO INVISIBLE

Al principio el juego se veía con un borde raro alrededor del mapa: el fondo por defecto de Godot (gris azulado) asomaba donde no llegaba el terreno. Lo arreglé poniendo `default_clear_color` en el celeste del cielo, así cualquier hueco del mapa queda del color correcto y no se nota.

3 El personaje y los controles

El jugador es lo primero que el usuario toca, así que es lo primero que tiene que sentirse bien. La idea fue simple: **cuatro direcciones, movimiento diagonal permitido, animación de dos cuadros y un único botón de acción**. Nada de combos ni de física complicada: caminar y apretar `E`.

3.1 El script completo, explicado

`player.gd` tiene 61 líneas. Es el script más corto del proyecto y el que más veces leí, porque cualquier error acá se siente en todo el juego.

player.gd · líneas 1-14 — declaraciones

```
extends CharacterBody2D

signal interact_requested(at_position: Vector2)

const SPEED := 92.0
const WORLD_SIZE := 5120.0
const EDGE_MARGIN := 20.0

@onready var sprite: Sprite2D = $Sprite2D

var facing_row := 0
var animation_clock := 0.0
var touch_directions := {"left": false, "right": false, "up": false, "down": false}
```

Cuatro decisiones que conviene poder justificar:

- `CharacterBody2D` y no `RigidBody2D`: no quiero física realista (inercia, rebotes). Quiero que el personaje se mueva exactamente a la velocidad que digo y que se frene al soltar la tecla.
- **La señal** `interact_requested`: el jugador *no sabe* qué hay al lado. Solo avisa "quiero interactuar acá". El mundo (`world.gd`) decide si eso es una radio, una tienda o un cultivo. Eso se llama **bajo acoplamiento**: puedo agregar objetos interactivos sin tocar el script del jugador.
- `SPEED = 92`: la calibré probando. Con 60 el personaje se sentía pesado; con 140, la cámara mareaba en el mapa grande. 92 píxeles por segundo en un mundo de 5.120 se siente caminable.
- `touch_directions`: un diccionario con las cuatro direcciones táctiles. En Android no hay teclado, así que los botones en pantalla escriben en este diccionario y el movimiento se calcula igual que con el teclado.

3.2 El movimiento

player.gd · líneas 16-36 movimiento

```
func _physics_process(delta: float) -> void:
        var horizontal := Input.get_axis("ui_left", "ui_right")
        var vertical := Input.get_axis("ui_up", "ui_down")
        if Input.is_key_pressed(KEY_A):
                horizontal -= 1.0
        if Input.is_key_pressed(KEY_D):
                horizontal += 1.0
        if Input.is_key_pressed(KEY_W):
                vertical -= 1.0
        if Input.is_key_pressed(KEY_S):
                vertical += 1.0
        horizontal += int(bool(touch_directions["right"])) - int(bool(touch_directions["left"]))
        vertical += int(bool(touch_directions["down"])) - int(bool(touch_directions["up"]))
        var direction := Vector2(horizontal, vertical)

        direction = direction.normalized()
        velocity = direction * SPEED
        move_and_slide()
        position = position.round()
        position.x = clampf(position.x, EDGE_MARGIN, WORLD_SIZE - EDGE_MARGIN)
        position.y = clampf(position.y, EDGE_MARGIN, WORLD_SIZE - EDGE_MARGIN)
```

Línea por línea, lo que importa:

1. `Input.get_axis()` devuelve −1, 0 o 1 según las acciones. Uso las acciones `ui_*` de Godot (flechas) y **sumo** WASD a mano con `is_key_pressed`. Así el jugador puede usar las dos cosas sin configurar un mapa de entrada duplicado.
2. `direction.normalized()`: sin esto, moverse en diagonal sería 1,41 veces más rápido (porque la diagonal de un cuadrado mide √2). Normalizar deja la velocidad pareja en las ocho direcciones. Es un error clásico y muy visible.
3. `move_and_slide()`: mueve el cuerpo y resuelve colisiones contra árboles, casas, el río y el granero. Es la razón de usar `CharacterBody2D`.
4. `position.round()`: redondeo la posición a píxeles enteros. En pixel art, una posición con decimales hace que el sprite “vibre” o se vea borroso. Este detalle no lo encontré en ningún tutorial: lo descubrí mirando el personaje temblar.
5. `clampf(...)`: límites del mundo. El jugador no puede salirse del mapa de 5.120 × 5.120. El margen de 20 px evita que quede pegado al borde exacto.

ERROR REAL #3 · LA DIAGONAL RÁPIDA

Mi primera versión no normalizaba la dirección. En diagonal el personaje avanzaba un 41% más rápido y se notaba muchísimo: los jugadores lo usaban para “correr”. Lo arreglé con `normalized()` una sola vez, antes de multiplicar por la velocidad.

3.3 La animación por cuadros

player.gd · líneas 38-47 — animación

```
	if direction != Vector2.ZERO:
		if absf(direction.x) > absf(direction.y):
			facing_row = 1 if direction.x < 0.0 else 2
		else:
			facing_row = 3 if direction.y < 0.0 else 0
		animation_clock += delta
		sprite.frame_coords = Vector2i(1 + int(animation_clock / 0.16) % 2, facing_row)
	else:
		animation_clock = 0.0
		sprite.frame_coords = Vector2i(0, facing_row)
```

El sprite del personaje es una **hoja de sprites** (una imagen con varios cuadros en grilla). Cada fila es una dirección —0 abajo, 1 izquierda, 2 derecha, 3 arriba— y cada columna es un cuadro de animación:

- Columna 0: quieto (un solo cuadro).
- Columnas 1 y 2: caminando (dos cuadros que se alternan).
- `animation_clock += delta` acumula tiempo real y `int(clock / 0.16) % 2` alterna entre 1 y 2 cada 0,16 segundos. Son unas 6 imágenes por segundo, que en pixel art se ve natural.
- La dirección se elige comparando `absf(x)` con `absf(y)`: si el movimiento es más horizontal, mira a los costados; si es más vertical, mira arriba o abajo.

No usé `AnimatedSprite2D` ni el `AnimationPlayer`. Podría, pero para cuatro filas y dos cuadros, cambiar `frame_coords` a mano es más directo y no necesita recursos de animación. Es una decisión de **simplicidad**: menos archivos, menos cosas que se pueden romper al exportar.

3.4 El botón de acción

player.gd · líneas 50-61 — input e interacción

```
func _input(event: InputEvent) -> void:
	if event is InputEventKey and event.pressed and not event.echo and event.keycode == KEY_E:
		request_interaction()


func set_touch_direction(direction_name: String, pressed: bool) -> void:
	if touch_directions.has(direction_name):
		touch_directions[direction_name] = pressed


func request_interaction() -> void:
	interact_requested.emit(global_position)
```

Tres detalles finos:

- `not event.echo`: impide que, si el jugador *mantiene* apretada la E, la acción se repita 30 veces por segundo. Sin esto, mantener E cosechaba y plantaba sin control.
- `event.keycode == KEY_E` en lugar de una acción del Input Map: quería que `E` fuera "usar" siempre, sin depender de configuración. Es más explícito y no se rompe si alguien edita el mapa de entrada.
- `global_position` en la señal: la interacción se decide por **distancia** desde la posición del jugador. Le paso la posición para que el mundo calcule a qué objeto está cerca.

3.5 Los controles táctiles

En Android el jugador no tiene teclado. La solución fue dibujar una cruceta y dos botones (Q y E) que escriben en el diccionario del jugador. `world.gd` los crea solo si detecta que está en un dispositivo táctil:

world.gd · líneas 324-337 — controles táctiles

```
func _build_touch_controls(force_mobile := false) -> void:
        if not force_mobile and not OS.has_feature("mobile") and not
DisplayServer.is_touchscreen_available():
                return
        touch_controls = Control.new()
        touch_controls.name = "TouchControls"
        touch_controls.set_anchors_and_offsets_preset(Control.PRESET_FULL_RECT)
        touch_controls.mouse_filter = Control.MOUSE_FILTER_IGNORE
        hud.add_child(touch_controls)
        _connect_direction_touch("↑", Vector2(36, 91), "up")
        _connect_direction_touch("←", Vector2(7, 119), "left")
        _connect_direction_touch("↓", Vector2(36, 119), "down")
        _connect_direction_touch("→", Vector2(65, 119), "right")
        _make_touch_button("Q", Vector2(230, 112), Vector2(34, 34)).pressed.connect(_mobile_cycle_seed)
        _make_touch_button("E", Vector2(270, 108), Vector2(42,
38)).pressed.connect(player.request_interaction)
```

La detección usa `OS.has_feature("mobile")` **o** `DisplayServer.is_touchscreen_available()`. La segunda condición es la que hace que en una notebook con pantalla táctil (o en la versión web abierta en un celular) también aparezcan los botones. Y el parámetro `force_mobile` me permitió probarlos en la PC sin publicar: lo llamaba a mano desde el editor.

POR QUÉ UN DICCIONARIO Y NO EVENTOS TÁCTILES DIRECTOS

Podría haber escuchado `InputEventScreenTouch` y mover al jugador desde ahí. Elegí el diccionario porque **unifica el movimiento**: el resto del código no sabe si el jugador usa teclado o pantalla, solo lee `touch_directions`. Eso también hace que la cruceta funcione igual en web con mouse.

PARA DECIR EN EL ORAL

"El jugador es intencionalmente tonto: solo se mueve, anima y avisa que quiere interactuar. Toda la lógica de qué hay cerca vive en el mundo. Esa separación me permitió sumar la radio, la tienda o la ruleta sin tocar una línea del script del personaje."

3.6 Qué probaría si tuviera que mejorarlo

Con más tiempo agregaría **inercia suave** (que el personaje acelere y frene en 0,1 s en vez de arrancar a velocidad plena) y **un guiño de feedback** al interactuar (un pequeño rebote del sprite). Son detalles chicos que hacen que el control se sienta menos "de robot". Los dejé afuera porque el objetivo era terminar el juego completo en cuatro plataformas, y esas horas rendían más en la radio y en la exportación.

4 El menú, los ajustes y la intro

Entre abrir el juego y jugar hay tres pantallas: el **menú**, la **intro** y la **transición** entre escenas. Las tres están hechas con código y las tres tienen una decisión de diseño detrás. Esta parte suele pasarse por alto, pero es la primera impresión: si el menú no se entiende, el juego ya empieza mal.



El menú principal. El fondo no es una imagen: está dibujado con rectángulos de color y recortes del tileset del pack (casa, árboles, pasto, camino de tierra).

4.1 El menú: una escena, tres vistas

El menú tiene tres estados: **inicio** (JUGAR o CONTINUAR + AJUSTES), **ajustes** (zoom de cámara) y **confirmación** (para cuando ya hay una partida y el jugador elige “Nueva partida”). En vez de hacer tres escenas, hice **tres contenedores dentro de la misma escena** y los muestro u oculto:

start_menu.gd · líneas 234-245 — cambio de vista

```
func _show_home() -> void:
        home_view.visible = true
        settings_view.visible = false
        confirm_view.visible = false


func _show_settings() -> void:
        _update_zoom_label()
        home_view.visible = false
        settings_view.visible = true
        confirm_view.visible = false
```

Es una máquina de estados minimalista. La ventaja: **no hay carga de escena** al pasar de una vista a otra (es instantáneo) y el estado del menú (el zoom elegido, por ejemplo) se mantiene vivo. La desventaja: si el menú creciera mucho, habría que separarlo en escenas. Para tres vistas, no valía la pena.

4.2 “JUGAR” o “CONTINUAR”: detectar si hay partida

El menú no puede ofrecer “Continuar” si no hay nada guardado. Esa consulta la resuelve el autoload `GameSave` con `has_save()`, y el menú arma los botones según la respuesta:

start_menu.gd · líneas 138-144 y 263-278 — lógica de partida

```
        var has_save := bool(get_node("/root/GameSave").call("has_save"))
        if has_save:
                home_view.add_child(_make_button("CONTINUAR", _continue_game, Vector2(138, 22)))
                home_view.add_child(_make_button("NUEVA PARTIDA", _start_game, Vector2(138, 22)))
        else:
                home_view.add_child(_make_button("JUGAR", _start_game, Vector2(138, 22)))


func _start_game() -> void:
        if bool(get_node("/root/GameSave").call("has_save")):
                home_view.visible = false
                settings_view.visible = false
                confirm_view.visible = true
                return
        _begin_new_game()


func _begin_new_game() -> void:
        get_node("/root/GameSave").call("clear_save")
        get_node("/root/SceneTransition").call("change_scene", INTRO_SCENE)


func _continue_game() -> void:
        get_node("/root/SceneTransition").call("change_scene", GAME_SCENE)
```

Hay una decisión importante acá: **“Nueva partida” pide confirmación** si ya hay progreso guardado, porque borra todo. Es un patrón de UX que copié de los juegos que jugué: nunca destruyas datos del usuario sin preguntar. El mensaje es explícito: *“Se borrará el progreso guardado.”*

4.3 Los ajustes: zoom de cámara que persiste

El único ajuste del menú es el **zoom de cámara** (de 0,75× a 2×, en pasos de 0,25). Lo puse porque en un celular chico el personaje se veía diminuto, y en un monitor grande se veía gigante. El ajuste se guarda en `settings.cfg` y se aplica al cargar el mundo:

```gdscript
game_settings.gd · líneas 1-35                                  ajustes persistentes

extends Node

const SETTINGS_PATH := "user://settings.cfg"
const MIN_CAMERA_ZOOM := 0.75
const MAX_CAMERA_ZOOM := 2.0
const CAMERA_ZOOM_STEP := 0.25

var camera_zoom := 1.0
var radio_volume := 0.75


func _ready() -> void:
        var config := ConfigFile.new()
        if config.load(SETTINGS_PATH) == OK:
                camera_zoom = clampf(
                        float(config.get_value("camera", "zoom", 1.0)),
                        MIN_CAMERA_ZOOM,
                        MAX_CAMERA_ZOOM
                )
                radio_volume = clampf(float(config.get_value("audio", "radio_volume", 0.75)), 0.0, 1.0)
```

Dos cosas que aprendí escribiendo esto:

- `clampf` **al leer, no solo al escribir.** Si alguien edita el archivo a mano y pone `zoom=50`, el juego no explota: lo limita al máximo. Los datos que vienen de disco *siempre* se validan.
- `snappedf(..., 0.25)` al escribir: el zoom siempre cae en un valor "redondo" (0,75 / 1 / 1,25…). Así el label muestra `1.25x` y no `1.2499999x`.



La vista de ajustes: el zoom se cambia con − y +, se ve el valor actual y se guarda solo. Es la misma pantalla que usé para probar la navegación con teclado (Tab y Enter).

ERROR REAL #4 · EL ZOOM NO SE APLICABA AL VOLVER

Al principio el menú cambiaba el valor, pero el mundo seguía con el zoom viejo. El problema: el `Camera2D` leía el zoom en su `_ready()` y yo cambiaba el ajuste después. Lo resolví aplicando el zoom en el `_ready()` de `world.gd`, que es donde la cámara ya existe:

```
$Player/Camera2D.zoom = Vector2.ONE * camera_zoom
```

4.4 La intro: contar la historia en 60 segundos

La intro son **cinco pantallas de 12 segundos** con un efecto de máquina de escribir. No es un video: es un `Control` con cinco textos y un temporizador. La escribí completa en un array, así cambiarla es editar un texto y nada más.

intro.gd · líneas 4-22 — duración y textos

```
const INTRO_DURATION := 60.0
const PAGE_DURATION := 12.0
const TYPEWRITER_SPEED := 34.0

const TITLES := [
        "LA CIUDAD",
        "UN NUEVO COMIENZO",
        "LA TIERRA ESPERA",
        "PRIMERAS COSECHAS",
        "UN LUGAR PROPIO",
]

const PAGES := [
        "Mateo llevaba diez años trabajando para el municipio. Sellaba papeles, cumplía horarios y cruzaba avenidas llenas de bocinas. Al terminar el día, el ruido seguía con él.",
        "No estaba escapando de su vida: necesitaba recuperar la calma. Cuando heredó una pequeña granja junto a un pueblo, guardó sus cosas, renunció a la oficina y tomó el camino hacia el campo.",
        "La granja estaba descuidada. El río partía el terreno, los surcos esperaban semillas y los vecinos tenían sus propios problemas. Mateo casi no sabía cultivar, pero estaba dispuesto a aprender.",
        "Con una azada y unas semillas empezó a trabajar. Una vieja radio le traía voces y música de Argentina mientras sembraba. Cada cosecha le daba recursos para mejorar la granja y ayudar a sus vecinos.",
        "Con el tiempo, la granja volvió a crecer y el pueblo recuperó su movimiento. Mateo se mudó buscando paz; también encontró un propósito. Ahora cada día empieza con una elección: ¿qué hará crecer hoy?",
]
```

Fijate que la **cuarta** pantalla es donde aparece la radio. No es casual: la radio no es un menú escondido, es parte de la historia. Ese texto justifica todo el capítulo 9.

intro.gd · líneas 40-55 el reloj de la intro

```
func _process(delta: float) -> void:
	if transition_started:
		return

	elapsed = minf(elapsed + delta, INTRO_DURATION)
	var page_index := mini(int(elapsed / PAGE_DURATION), PAGES.size() - 1)
	if page_index != current_page:
		_show_page(page_index)

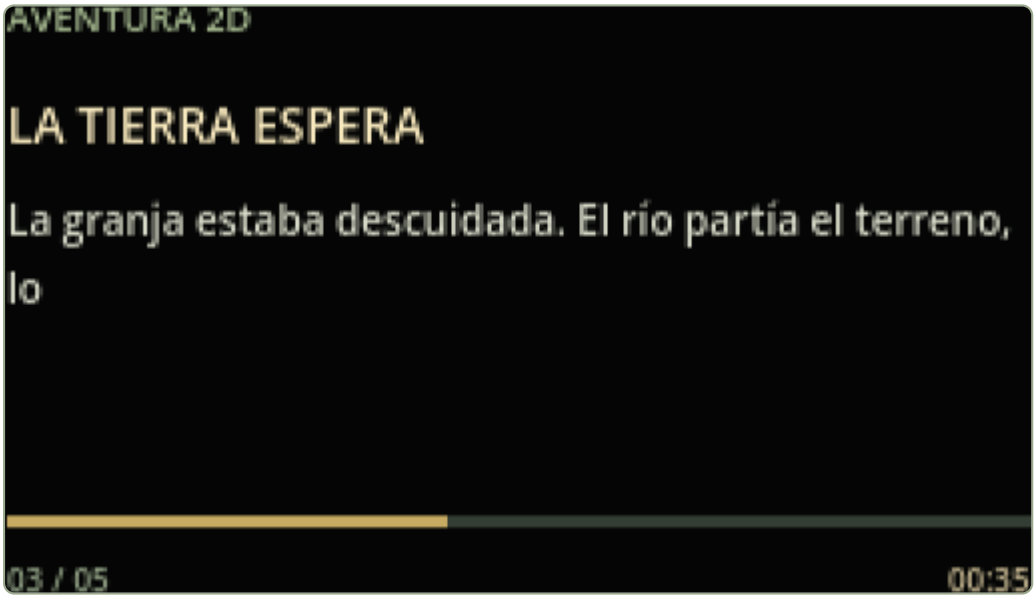
	var page_elapsed := minf(elapsed - float(current_page) * PAGE_DURATION, PAGE_DURATION)
	story_text.visible_characters = mini(PAGES[current_page].length(), int(page_elapsed *
TYPEWRITER_SPEED))
	_update_time_and_progress()

	if elapsed >= INTRO_DURATION:
		transition_started = true
		get_node("/root/SceneTransition").call("change_scene", GAME_SCENE)
```

El truco de la máquina de escribir es `visible_characters`: el texto completo ya está en el label, pero solo se dibujan los primeros N caracteres. N se calcula como *tiempo transcurrido* × *34*. Es más eficiente que ir concatenando strings, y no tiene problemas con los acentos.

POR QUÉ 5 × 12 = 60 SEGUNDOS

Lo medí leyendo en voz alta: 12 segundos alcanzan para leer cada párrafo dos veces. Si la intro es más larga, el jugador quiere saltearla; si es más corta, no se entiende la historia. Además, con 60 segundos exactos el cálculo es trivial: **página = tiempo / 12**.



La pantalla 3 de 5 (“La tierra espera”), con el texto escribiéndose, el contador de 60 segundos y la barra de progreso abajo.

4.5 La transición: un shader de píxeles

Cambiar de escena con un corte seco se siente barato. Quería algo que dijera “estás en un juego de píxeles”, así que escribí un shader que disuelve la pantalla en bloques. La transición es un `ColorRect` a pantalla completa con un material que anima un parámetro `progress` de 0 a 1.

scene_transition.gd · líneas 26-39 — secuencia de la transición

```gdscript
func change_scene(scene_path: String) -> void:
	if transitioning:
		return
	transitioning = true
	overlay.visible = true
	transition_material.set_shader_parameter("progress", 0.0)

	var tween := create_tween()
	tween.set_trans(Tween.TRANS_LINEAR)
	tween.tween_method(_set_progress, 0.0, 1.0, FADE_SECONDS)
	tween.tween_callback(_load_scene.bind(scene_path))
	tween.tween_interval(0.12)
	tween.tween_method(_set_progress, 1.0, 0.0, FADE_SECONDS)
	tween.tween_callback(_finish_transition)
```

La secuencia es una **línea de tiempo** de cuatro pasos: cubrir (0,42 s) → cambiar la escena → esperar 0,12 s → descubrir (0,42 s). El pequeño intervalo del medio es importante: da tiempo a que la escena nueva termine de cargar antes de mostrarse.

Y hay un detalle defensivo: `if transitioning: return`. Si el jugador aprieta dos veces el botón, la segunda llamada se ignora. Sin eso, se disparaban dos transiciones superpuestas y la escena quedaba en negro.



La transición a mitad de camino: los bloques de píxeles cubren la pantalla mientras se carga la escena siguiente. Dura menos de un segundo, pero es lo que hace que el juego se sienta "terminado".

PARA DECIR EN EL ORAL

"La transición no es un adorno: es la que me permite cambiar de escena sin que se vea el salto. Es un autoload, un ColorRect a pantalla completa con un shader, y una animación de cuatro pasos con Tween. Si no fuera autoload, desaparecería junto con la escena vieja y el cambio se vería."

ERROR REAL #5 · LA ESCENA EN NEGRO

Al principio la transición tapaba la pantalla y nunca la descubría: usaba `await get_tree().create_timer(...)` después de cambiar de escena, y el `await` quedaba en un nodo que ya no existía en el árbol nuevo. Lo resolví con `Tween`, que está atado al nodo autoload y sobrevive al cambio porque el autoload no se destruye.

5 La granja: cultivos, crecimiento y cosecha

Este es el corazón del juego. Todo lo demás (tienda, niveles, radio, clima) existe alrededor de este ciclo: **plantar, esperar, cosechar**. Si esta parte no se siente bien, el juego no funciona, por más bonito que sea el resto.

5.1 Cómo están guardados los cultivos

Podría haber hecho una clase `Crop` con sus propiedades. En cambio usé **arrays paralelos**: seis arrays donde el índice es el número de surco. Es menos "elegante" pero mucho más simple de guardar y de recorrer.

world.gd · líneas 99-106 — los arrays de cultivos

```
var crops: Array[Vector2] = []            # posición en el mundo de cada surco
var crop_sprites: Array[Sprite2D] = []    # el sprite que lo dibuja
var regrow_timers: Array[float] = []      # segundos que le faltan para madurar
var crop_planted: Array[bool] = []        # ¿hay algo plantado?
var crop_types: Array[int] = []           # 0 trigo, 1 zanahoria, 2 tomate
var crop_growth_seconds: Array[float] = [] # tiempo total de crecimiento
```

La regla es: **el índice `i` representa el mismo surco en los seis arrays**. Si `crop_planted[7]` es `true`, entonces `crop_types[7]` dice qué hay plantado, `regrow_timers[7]` cuánto falta y `crop_sprites[7]` es el sprite que lo muestra. Esa convención es la que hace que todo el resto funcione.

Cultivo	Tiempo	Recompensa	XP	Por qué esos valores
Trigo	90 s	10 fichas	5	El más lento y el más barato: es el cultivo "de arranque".
Zanahoria	60 s	15 fichas	10	Más rápida y mejor paga: premia al que compra semillas mejores.
Tomate	75 s	15 fichas	15	El que más XP da: es el camino para subir de nivel.

POR QUÉ ARRAYS PARALELOS Y NO OBJETOS

Con objetos (`class Crop`) el código se lee más lindo. Pero guardar en `ConfigFile` un array de objetos obliga a serializar cada uno. Con arrays paralelos, el guardado es `crop_planted.duplicate()` y listo: son tipos simples (bool, int, float) que el archivo guarda sin conversiones. **Menos código de serialización, menos bugs.**

5.2 Plantar

world.gd · líneas 1144-1165 _plant_crop()

```
func _plant_crop(index: int) -> void:
	if not extra_land_unlocked:
		message_label.text = "Comprá las parcelas nuevas en la tienda para plantar ahí."
		return
	var crop_type := selected_crop_type
	if seed_inventory[crop_type] <= 0:
		_cycle_crop_seed()
		crop_type = selected_crop_type
	if seed_inventory[crop_type] <= 0:
		message_label.text = "No quedan semillas para esta huerta."
		return
	crop_types[index] = crop_type
	crop_growth_seconds[index] = CROP_GROWTH_SECONDS[crop_type]
	crop_sprites[index].texture = _make_crop_atlas(crop_type)
	crop_sprites[index].modulate = Color.WHITE
	crop_planted[index] = true
	seed_inventory[crop_type] -= 1
	regrow_timers[index] = crop_growth_seconds[index]
	crop_sprites[index].visible = true
	_set_crop_growth(index, 0.0)
	message_label.text = "Plantaste %s (%d semillas). Crece en %s." %
[CROP_NAMES[crop_type].to_lower(), seed_inventory[crop_type], _format_time(crop_growth_seconds[index])]
	_update_controls_hint()
```

La función hace cinco cosas, en orden: **valida** (¿tengo tierra?, ¿tengo semillas?), **elige** el cultivo, **actualiza los seis arrays**, **descuenta la semilla** y **avisa al jugador**. Ese “avisa al jugador” es una línea de texto en el HUD, y es más importante de lo que parece: sin feedback, el jugador no sabe si la acción funcionó.

Hay un detalle de comodidad: si no tenés semillas del cultivo seleccionado, **cambia solo** al siguiente que sí tenga (`_cycle_crop_seed()`). Es una decisión de UX: evita que el jugador apriete E y no pase nada sin entender por qué.

5.3 Crecer: el temporizador y el clima

El crecimiento no usa un `Timer` por cultivo (serían decenas de nodos). Usa un **único bucle** en el `_process()` del mundo que descuenta tiempo a todos los surcos a la vez, multiplicado por el clima:

world.gd · líneas 253-267 el bucle de crecimiento

```
func _process(delta: float) -> void:
	autosave_clock += delta
	if autosave_clock >= AUTOSAVE_SECONDS:
		autosave_clock = 0.0
		_save_progress()
	weather_timer -= delta
	if weather_timer <= 0.0:
		_change_weather()
	var growth_delta: float = delta * WEATHER_GROWTH_MULTIPLIERS[weather_type]
	for index in range(regrow_timers.size()):
		if regrow_timers[index] > 0.0:
			regrow_timers[index] = maxf(0.0, regrow_timers[index] - growth_delta)
			crop_sprites[index].visible = true
			var growth := 1.0 - regrow_timers[index] / crop_growth_seconds[index]
			_set_crop_growth(index, growth)
```

Ese `growth_delta` es la línea más importante del sistema: el tiempo que avanza cada cultivo depende del clima. Con lluvia (`1.6`) crecen un 60% más rápido; con sequía (`0.6`) un 40% más lento. Así el clima no es

decorativo: **cambia la economía del juego**.

Y `_set_crop_growth(index, growth)` traduce ese 0..1 a un **cuadro de la animación** del cultivo: 0 es semilla, 0,5 es planta, 1 es planta lista. El arte del pack ya trae los cuadros; yo solo elijo cuál mostrar según el porcentaje.

5.4 Cosechar (y el trigo dorado)

world.gd · líneas 1198-1227 _harvest_crop()

```
func _harvest_crop(index: int, automated: bool) -> void:
	if not crop_planted[index] or regrow_timers[index] > 0.0:
		return
	var crop_type := crop_types[index]
	var reward: int = CROP_HARVEST_REWARDS[crop_type]
	var xp_reward: int = CROP_XP_REWARDS[crop_type]
	harvest_count += 1
	regrow_timers[index] = crop_growth_seconds[index]
	crop_sprites[index].visible = true
	_set_crop_growth(index, 0.0)
	coins += reward
	var found_golden := false
	if crop_type == CROP_WHEAT and not golden_found and randi_range(1, golden_wheat_odds) == 1:
		golden_found = true
		golden_crop_index = index
		regrow_timers[index] = 0.0
		crop_sprites[index].visible = true
		_set_crop_growth(index, 1.0)
		crop_sprites[index].modulate = Color("fff07a")
		coins += 100
		found_golden = true
	if automated:
		message_label.text = "Un empleado cosechó %s (+%d fichas)." % [CROP_NAMES[crop_type],
reward]
	else:
		message_label.text = "Cosecha %d de %s: +%d XP, +%d fichas." % [harvest_count,
CROP_NAMES[crop_type], xp_reward, reward]
	if found_golden:
		message_label.text = "¡TRIGO DORADO! +100 fichas; cosecha %d." % harvest_count
	_award_xp(xp_reward)
	_update_status()
	_update_shop_ui()
```

La cosecha es **automática al apretar E** sobre un surco maduro, pero el surco no desaparece: vuelve a crecer. El juego es un ciclo infinito, no un inventario que se vacía. Esa decisión es la que hace que puedas jugar diez minutos o dos horas con el mismo campo.

El trigo dorado

Quería un “premio raro” que hiciera que cosechar siempre tuviera una sorpresa. El trigo dorado tiene una probabilidad de **1 en 2.000.000.000** (sí, dos mil millones) y da 100 fichas extra. Suena imposible, y lo es… hasta que comprás mejoras de suerte en la tienda:

world.gd · líneas 5 y 21-23 probabilidades por mejora

```
const GOLDEN_WHEAT_ODDS := 2_000_000_000
const MAX_LUCK_UPGRADES := 5
const LUCK_ODDS_BY_UPGRADE := [2_000_000_000, 20_000_000, 500_000, 10_000, 1_000, 250]
const LUCK_UPGRADE_COSTS := [250, 500, 1_000, 2_000, 3_500]
```

Con las cinco mejoras compradas, la probabilidad pasa a **1 en 250**: una cada 250 cosechas de trigo. Eso convierte a la suerte en una **inversión a largo plazo**, que es exactamente lo que hace que un juego de granja enganche.

SOBRE `RANDI_RANGE(1, 2_000_000_000)`

La comparación es `randi_range(1, odds) == 1`: hay exactamente una posibilidad entre *odds* de que salga 1. Es más claro que dividir por una probabilidad en flotante y evita errores de redondeo. Y como el trigo dorado **solo puede aparecer una vez** (`not golden_found`), es un logro único de la partida.

PARA DECIR EN EL ORAL

"El clima no es decorativo: multiplica la velocidad de crecimiento. Un cultivo de trigo tarda 90 segundos, pero con lluvia tarda 56. Eso hace que el jugador mire el clima antes de decidir qué plantar, y le da una razón para quedarse jugando cuando llueve."

ERROR REAL #6 · LOS CULTIVOS FLORECÍAN EN EL AIRE

Al recargar la partida, algunos surcos aparecían sin planta aunque estaban plantados. El bug: restauraba `regrow_timers` pero no volvía a poner `crop_sprites[i].visible = true` cuando el temporizador era mayor que cero. Lo arreglé en `_restore_progress()` y después agregué una prueba manual: guardar a mitad de crecimiento, cerrar, abrir y verificar que las plantas siguen ahí.

6 Economía, tienda y empleados

Un juego de granja sin economía es un simulador de esperar. Lo que hace que el jugador vuelva es la sensación de que **cada cosecha lo acerca a algo**. Para eso armé dos monedas que se ganan distinto y se gastan distinto: las **fichas** y la **experiencia**.

6.1 Las dos monedas

	Fichas (dinero)	XP (experiencia)
Se gana	Cosechando, subiendo de nivel, en la ruleta	Cosechando
Se usa para	Comprar mejoras en la tienda	Subir de nivel (automático)
Se pierde	Al apostar en la ruleta	Nunca
Qué desbloquea	Poder económico	Contenido (nivel 5 granero, nivel 10 ruleta, nivel 50 sector nuevo)

Esa separación es intencional. Si el nivel se comprara con fichas, el jugador con más plata sería el más avanzado y el juego se rompería. Al ser la XP **solo por trabajar**, el progreso siempre está atado a jugar.

6.2 La curva de niveles

world.gd · líneas 1323-1370 — XP, niveles y desbloqueos

```
func _award_xp(amount: int) -> void:
	var old_level := level
	total_xp += amount
	var progress := _level_progress_for_total_xp(total_xp)
	level = int(progress[0])
	_update_building_locks()
	if level > old_level:
		var levels_gained := level - old_level
		var level_bonus := levels_gained * 5
		coins += level_bonus
		var unlock_messages: Array[String] = []
		if old_level < 5 and level >= 5:
			unlock_messages.append("se abrió el granero de duelos")
		if old_level < 10 and level >= 10:
			unlock_messages.append("se abrió la ruleta")
		if old_level < 50 and level >= 50:
			_open_zone_gate()
			unlock_messages.append("se abrió el nuevo sector")
		if unlock_messages.is_empty():
			message_label.text = "¡Subiste al nivel %d! +%d fichas de recompensa." % [level,
level_bonus]
		else:
			message_label.text = "¡Nivel %d! +%d fichas: %s." % [level, level_bonus, ",
".join(PackedStringArray(unlock_messages))]


func _xp_required_for_level(current_level: int) -> int:
	if current_level < 50:
		return XP_PER_LEVEL
	var difficulty_steps := int((current_level - 50) / 10) + 1
	return XP_PER_LEVEL + difficulty_steps * 100
```

La curva tiene dos tramos:

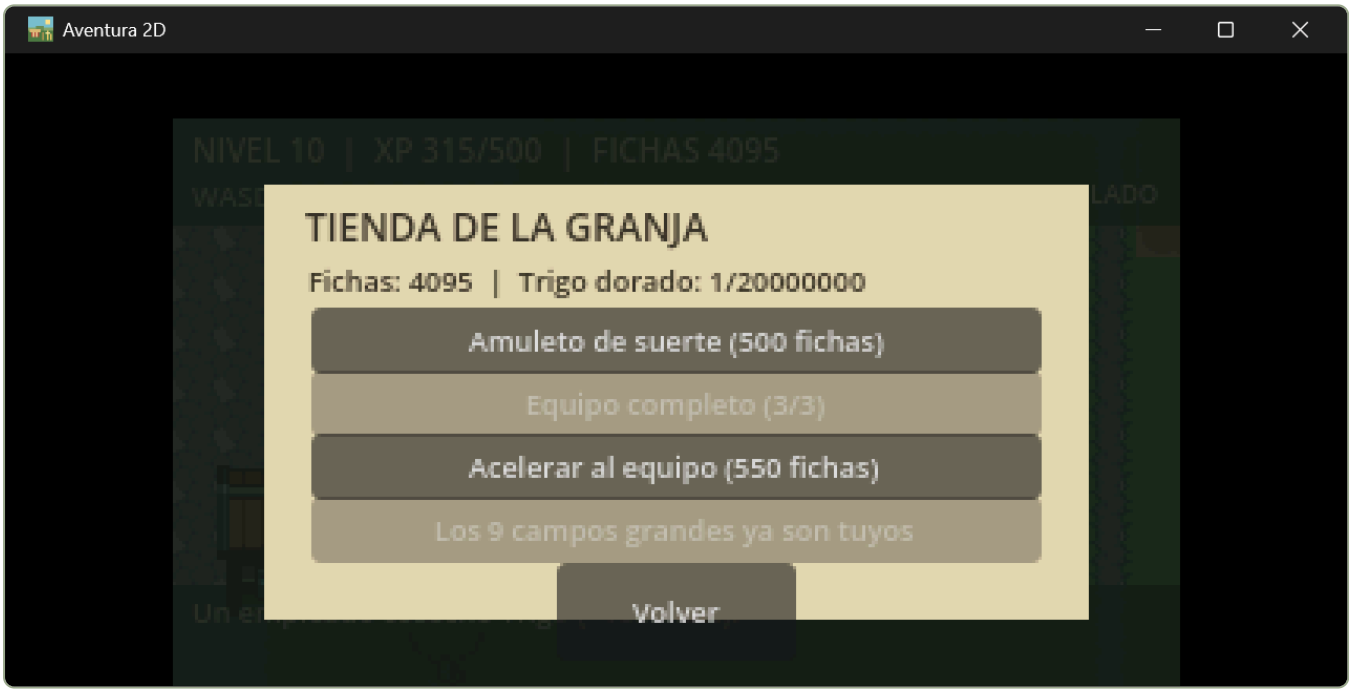
- **Del nivel 1 al 50:** 500 XP fijos por nivel. Es una curva plana y generosa: el jugador siente que avanza. Con el tomate (15 XP por cosecha) son unas 34 cosechas por nivel.
- **Desde el 50:** cada 10 niveles la XP necesaria sube 100. El nivel 50 es el hito del "sector nuevo", así que a partir de ahí el juego se vuelve más exigente para que llegar al 60 signifique algo.

Y hay un detalle que suena menor pero no lo es: **cada nivel da 5 fichas**. Es una recompensa chica, pero hace que subir de nivel se sienta doble: desbloqueás contenido *y* cobrás.

6.3 La tienda

La tienda tiene cuatro cosas para comprar, y cada una resuelve un problema distinto del jugador:

Mejora	Costo	Qué hace	Problema que resuelve
Amuleto de suerte	250 / 500 / 1.000 / 2.000 / 3.500	Baja la probabilidad del trigo dorado hasta 1 en 250	Da una razón para cosechar mucho trigo
Empleados	500 / 900 / 1.500	Contrata hasta 3 trabajadores que cosechan solos	El jugador no tiene que estar en cada surco
Velocidad del equipo	300 / 550 / 850 / 1.200	Cada empleado se mueve 18 px/s más rápido	Escala la producción sin agregar más empleados
9 campos grandes	2.500	Abre una zona nueva con 9 campos y 120 semillas de cada cultivo	El salto de escala del juego



La tienda abierta en el juego. Arriba las fichas actuales y el contador de trigo dorado; cada botón se deshabilita solo si no alcanza el dinero (por eso "Acelerar al equipo" aparece en gris).

world.gd · líneas 2113-2144 — comprar suerte y contratar empleados

```
func _buy_luck_upgrade() -> void:
	if luck_upgrades >= MAX_LUCK_UPGRADES:
		return
	var cost: int = LUCK_UPGRADE_COSTS[luck_upgrades]
	if coins < cost:
		return
	coins -= cost
	luck_upgrades += 1
	golden_wheat_odds = LUCK_ODDS_BY_UPGRADE[luck_upgrades]
	_update_status()
	_update_shop_ui()
	message_label.text = "Amuleto comprado: ahora es 1 entre %s." % _format_number(golden_wheat_odds)
	_save_progress()


func _buy_worker() -> void:
	if employee_count >= MAX_EMPLOYEES:
		return
	var cost: int = EMPLOYEE_HIRING_COSTS[employee_count]
	if coins < cost:
		return
	var new_employee_index := employee_count
	coins -= cost
	employee_count += 1
	worker_hired = true
	worker_sprites[new_employee_index].visible = true
	worker_sprites[new_employee_index].position = WORKER_HOME_POSITION + Vector2((new_employee_index %
2) * 12, int(new_employee_index / 2) * 12)
	worker_targets[new_employee_index] = -1
	_update_status()
	_update_shop_ui()
	message_label.text = "Contrataste a %s. Tu equipo ya tiene %d empleados." %
[FARMHAND_NAMES[new_employee_index], employee_count]
	_save_progress()
```

Las tres validaciones del principio son el patrón que se repite en **toda** la tienda: *¿está al máximo? → salir*, *¿alcanza el dinero? → salir*, *cobrar, aplicar el efecto, avisar y guardar*. Es simple y no tiene agujeros: aunque el botón estuviera habilitado por error, la función se protege sola.

DETALLE DE UX: EL BOTÓN DESHABILITADO

`shop_luck_button.disabled = coins < cost`. El botón se apaga cuando no alcanza el dinero, **pero la lógica de compra igual valida**. Es defensa en capas: la interfaz informa y el código protege. Nunca confío solo en el estado visual.

ERROR REAL #7 · EL EMPLEADO INVISIBLE QUE TRABAJABA

Al contratar al segundo empleado, el sprite se creaba pero no se veía: lo posicionaba exactamente sobre el primero. Lo arreglé desplazándolos en grilla con `(índice % 2) * 12` y `int(índice / 2) * 12`, así cada uno aparece al lado del otro. Y como el trabajo lo hace la lógica (no el sprite), el empleado "invisible" igual cosechaba: ese fue el dato que me hizo entender que el problema era solo visual.

6.4 Los empleados: trabajo automático

Los tres empleados (Mora, Tomi y Rosa) recorren el campo, buscan el cultivo maduro más cercano y lo cosechan. No son decorativos: **cada cosecha de un empleado paga lo mismo que la del jugador**,

pero sin XP. Esa diferencia mantiene el equilibrio: el dinero se automatiza, el progreso no.

world.gd · líneas 1235-1263 — lógica de un empleado

```
func _process_worker(delta: float) -> void:
	if employee_count <= 0:
		return
	for worker_index in range(mini(employee_count, worker_sprites.size())):
		var target_index := worker_targets[worker_index]
		if target_index >= 0 and (not crop_planted[target_index] or regrow_timers[target_index] >
0.0):
			worker_targets[worker_index] = -1
			target_index = -1
		if target_index < 0:
			target_index = _find_worker_target(worker_index)
			worker_targets[worker_index] = target_index
		if target_index < 0:
			continue

		var farmhand := worker_sprites[worker_index]
		var offset := crops[target_index] - farmhand.position
		if offset.length() <= worker_speed * delta:
			farmhand.position = crops[target_index]
			_harvest_crop(target_index, true)
			worker_targets[worker_index] = -1
			continue

		var direction := offset.normalized()
		farmhand.position += direction * worker_speed * delta
```

El patrón es: **si el objetivo dejó de ser válido, lo suelto; si no tengo objetivo, busco uno; si ya llegué, cosecho; si no, me muevo**. Cada empleado tiene su propio `worker_targets[i]`, así no se pelean por el mismo surco.

La condición `offset.length() <= worker_speed * delta` es la que evita que el empleado "vibre" sobre el cultivo: si en este cuadro va a pasar de largo, se teletransporta al objetivo y cosecha. Es un truco común en movimiento por cuadros y se nota muchísimo cuando falta.

> PARA DECIR EN EL ORAL
>
> *"La economía tiene dos monedas a propósito: las fichas compran comodidad y la experiencia compra progreso. Si el dinero diera niveles, el juego se resolvería pagando. Como la XP solo sale de cosechar, el jugador que avanza es el que juega."*

6.5 Los otros dos edificios

Además de la tienda hay dos lugares que se desbloquean por nivel:

- **Granero de duelos (nivel 5):** un combate por turnos simple contra un enemigo. Tiene atacar, curar (una vez) y huir. Sirve como cambio de ritmo: no todo es cosechar.
- **Ruleta (nivel 10):** apuestas fichas a un número. Si sale, multiplicás; si no, perdés. Es el único lugar del juego donde podés perder dinero, y por eso está bloqueada hasta el nivel 10: quiero que el jugador ya tenga una economía estable antes de arriesgarla.

POR QUÉ LA RULETA PAGA CON NÚMEROS ROJOS Y NEGROS

`const RED_NUMBERS := [1, 3, 5, 7, 9, 12, 14, 16, 18, 19, 21, 23, 25, 27, 30, 32, 34, 36]`. Usé la rueda europea real (18 rojos, 18 negros y el 0 verde). No es capricho: si inventaba las reglas, el jugador podía quejarse de que “estaba arreglado”. Con la rueda real, la probabilidad es la misma que en un casino y se puede explicar.

7 El mundo: mapa, clima y vida

Si los cultivos son el corazón, el mundo es el cuerpo. Necesitaba que el mapa se sintiera **grande pero no vacío**, y que tuviera vida propia: un río que corta el terreno, un pueblo, animales, clima que cambia. Todo eso lo construí **por código**, no con un editor de mapas externo.



Una partida real: arriba el HUD con nivel, XP, fichas y clima (NUBLADO); abajo el mensaje de un empleado que cosechó trigo; en el medio la tienda, la radio, el granero bloqueado y el campo.

7.1 El tamaño del mundo y sus zonas

world.gd · líneas 3-4 y 32-49 — coordenadas del mundo

```
const WORLD_SIZE := 5120
const WORLD_CENTER := Vector2(2048, 2048)

const BATTLE_BARN_POSITION := Vector2(1963, 2012)
const ROULETTE_HUT_POSITION := Vector2(2014, 2168)
const SHOPKEEPER_POSITION := Vector2(1918, 2120)
const WORKER_HOME_POSITION := Vector2(1878, 2118)
const FIELD_TOP_LEFT := Vector2(2070, 1968)
const RIVER_X := 2360.0
const RIVER_WIDTH := 96.0
const RIVER_BRIDGE_Y := 2048.0
const ZONE_GATE_POSITION := Vector2(2440, 2048)
const NEW_ZONE_SPAWN := Vector2(3100, 2048)
const VILLAGE_CENTER := Vector2(1490, 2048)
const RADIO_POSITION := Vector2(1950, 2170)
```

Zona	Dónde	Qué hay
Granja principal	centro (2048, 2048)	Campo de cultivos, tienda, radio, granero, choza de la ruleta
Pueblo La Tranquera	oeste (1490, 2048)	Vecino con diálogos rotativos y casas del pack
Río	x = 2360	Corta el mapa de norte a sur; se cruza por dos puentes
Sector nuevo	este (3100, 2048)	9 campos grandes; se abre en el nivel 50
Bosques	grilla de 4 × 4	Árboles con colisión, generados con posición aleatoria
Lago	(1340..2020, 1390..1875)	Agua, borde de arena y un puente propio

El mapa no está "pintado" a mano: está **descripto por constantes**. Eso tiene una ventaja enorme: para mover la tienda, cambio un número y listo. Y todas las distancias de interacción (25-32 px) están calculadas contra esas mismas constantes, así que nada queda desalineado.

7.2 El clima

El clima tiene cuatro estados y cada uno modifica el crecimiento. Cambia solo, cada 65-105 segundos, y **nunca repite el mismo** dos veces seguidas:

world.gd · líneas 24-31 y 381-388 — clima

```
const WEATHER_NAMES := ["SOLEADO", "LLUVIOSO", "SEQUÍA", "NUBLADO"]
const WEATHER_GROWTH_MULTIPLIERS := [1.0, 1.6, 0.6, 0.85]
const WEATHER_DURATION_MIN := 65.0
const WEATHER_DURATION_MAX := 105.0


func _change_weather() -> void:
	var next_weather := randi_range(0, WEATHER_NAMES.size() - 2)
	if next_weather >= weather_type:
		next_weather += 1
	weather_type = next_weather
	weather_timer = randf_range(WEATHER_DURATION_MIN, WEATHER_DURATION_MAX)
	_update_weather_visuals()
	message_label.text = "Cambió el clima: %s. El crecimiento se ajustó." % WEATHER_NAMES[weather_type]
	_save_progress()
```

Ese truco de `randi_range(0, size - 2)` + `if next >= current: next += 1` es una forma elegante de elegir un elemento **distinto** al actual sin bucles ni reintentos. Lo uso porque el clima repitiéndose se siente trabado.

Lo visual también cambia, y no solo el cartel:

world.gd · líneas 392-406 — tinte y lluvia

```
func _update_weather_visuals() -> void:
	weather_label.text = "CLIMA: %s" % WEATHER_NAMES[weather_type]
	var tint_colors := [
		Color(0.0, 0.0, 0.0, 0.0),      # soleado: sin tinte
		Color(0.12, 0.2, 0.34, 0.18),   # lluvioso: azul
		Color(0.45, 0.27, 0.06, 0.13),   # sequía: ámbar
		Color(0.24, 0.27, 0.32, 0.1),    # nublado: gris
	]
	weather_tint.color = tint_colors[weather_type]
	weather_rain.emitting = weather_type == WEATHER_RAINY
```

La lluvia es un `CPUParticles2D` con 72 partículas de 2 × 8 píxeles blancos, gravedad hacia abajo y una inclinación leve. No usé una imagen de gota: generé el texture en memoria con `Image.create(2, 8, ...)`. Es el ejemplo más claro de que en pixel art **no hace falta arte para todo**.

> **POR QUÉ EL TINTE VA EN EL HUD Y NO EN EL MUNDO**
>
> El `ColorRect` del clima es hijo del `CanvasLayer` y se dibuja **encima de todo**. Si lo pusiera en el mundo, se movería con la cámara y habría que ajustarlo cada cuadro. Puesto en el HUD, cubre la pantalla siempre, sin importar dónde esté el jugador.

7.3 La vida del mapa

Un mapa quieto se siente muerto. Agregué tres capas de vida, todas baratas en recursos:

1. **Gaviota:** un `CharacterBody2D` que orbita el centro del mapa en una elipse (radios 1500 × 1100) a 55 px/s. No interactúa con nada: solo cruza el cielo cada tanto y hace que el mundo se sienta habitado.
2. **Conejo:** aparece cerca de la granja, espera, salta a un punto aleatorio y vuelve. Tiene su propio temporizador y destino.
3. **Animales ambientales y vendedor:** comparten un reloj de animación de 0,28 s y alternan tres cuadros. Una sola línea mueve a todos.

world.gd · líneas 409-420 — la gaviota en órbita

```
func _physics_process(delta: float) -> void:
	if not is_instance_valid(gull_body):
		return

	gull_orbit_angle = wrapf(gull_orbit_angle + GULL_ORBIT_SPEED * delta, -PI, PI)
	var target := WORLD_CENTER + Vector2(
		cos(gull_orbit_angle) * GULL_ORBIT_RADII.x,
		sin(gull_orbit_angle) * GULL_ORBIT_RADII.y
	)
	var direction := target - gull_body.position
	if direction.length_squared() <= 1.0:
		return
```

La elipse sale de multiplicar coseno y seno por radios distintos: con el mismo radio sería un círculo. Y `wrapf(ángulo, -PI, PI)` evita que el ángulo crezca sin límite durante horas de juego (un problema real en partidas largas).

7.4 La migración de coordenadas

Un problema que no anticipé: cuando agregué los 9 campos grandes, el índice de cada surco cambió. Si el jugador tenía una partida guardada, sus cultivos aparecían en surcos equivocados. La solución fue **versionar el guardado** y migrar:

world.gd · líneas 1435-1445 — migración de índices

```
	var has_saved_crop_layout := state.has("crop_planted")
	var saved_field_layout_version := int(state.get("field_layout_version", 1))
	...
	for saved_index in range(saved_crop_count):
		var crop_index := _migrate_saved_crop_index(saved_index, has_saved_crop_layout,
saved_field_layout_version)
		if crop_index >= crops.size():
			continue
```

Cuando el guardado no tiene `field_layout_version`, sé que es de la versión vieja y aplico la traducción de índices. Es la misma idea que usé con `economy_version` para los niveles: **un guardado viejo tiene que seguir abriendo**, aunque el juego haya cambiado.

PARA DECIR EN EL ORAL

"El mapa entero está definido por constantes: posiciones, río, puentes, zonas. Eso me permitió cambiar el diseño sin romper las distancias de interacción. Y cuando agregué campos nuevos, versioné el guardado para que las partidas viejas no se corrompieran: un juego que actualiza tiene que respetar el progreso del jugador."

ERROR REAL #8 · LA GAVIOTA ATRAVESABA LAS CASAS

La gaviota es un cuerpo con colisión y al orbitar chocaba con los árboles del borde, se trababa y quedaba vibrando. Lo resolví sacándole la colisión con el escenario (solo se mueve en una capa superior) y haciéndola "volar": no colisiona con nada, porque visualmente está por encima de todo.

8 El guardado: que no se pierda nada

Este capítulo lo escribo en serio: el guardado es lo que separa un prototipo de un juego. Si el jugador pierde dos horas de progreso, no vuelve. Yo lo aprendí de la peor manera: la primera versión guardaba solo las fichas, y al cerrar el juego **los cultivos volvían a estar vacíos**.

8.1 Dónde y cómo se guarda

Godot tiene una carpeta especial por aplicación: `user://`. En Windows es `%APPDATA%\Godot\app_userdata\Aventura 2D\`, en Android es el almacenamiento interno de la app y en iOS su carpeta de documentos. Ahí van dos archivos:

Archivo	Tamaño	Qué guarda
`savegame.cfg`	~9,6 KB	Todo el progreso: nivel, XP, fichas, cultivos, clima, posición del jugador, empleados…
`settings.cfg`	~46 bytes	Zoom de cámara y volumen de radio. Son preferencias, no progreso.

Que sean **dos archivos separados** es a propósito: borrar la partida no tiene que resetear el volumen ni el zoom. Son cosas distintas y se administran distinto.

El formato es `ConfigFile`, que es texto plano con secciones:

savegame.cfg · extracto real — formato del guardado

```
[progress]

state={
"coins": 3895,
"crop_growth_seconds": PackedFloat32Array(...),
"crop_planted": [...],
"economy_version": 2,
"employee_count": 2,
"field_layout_version": 2,
"harvest_count": 214,
"level": 10,
"player_position": Vector2(1935, 2109),
"weather_type": 3,
"weather_timer": 42.5,
...
}
```

POR QUÉ TEXTO Y NO UN ARCHIVO BINARIO

Un binario es más chico y “más pro”, pero un `.cfg` lo podés abrir con el Bloc de notas y ver qué está mal. Durante el desarrollo eso me salvó varias veces: podía verificar si el problema era el guardado o la restauración. Además pesa 9 KB: el tamaño no era un problema.

8.2 El autoload GameSave

Toda la lectura y escritura pasa por un solo lugar. Eso significa que si mañana cambio el formato, cambio **un archivo**:

game_save.gd · líneas 1-33 (completo) el servicio de guardado

```gdscript
extends Node

const SAVE_PATH := "user://savegame.cfg"
const SAVE_VERSION := 1


func has_save() -> bool:
	return FileAccess.file_exists(SAVE_PATH) and not load_state().is_empty()


func load_state() -> Dictionary:
	var config := ConfigFile.new()
	if config.load(SAVE_PATH) != OK:
		return {}
	var state: Variant = config.get_value("progress", "state", {})
	if not state is Dictionary or int(state.get("version", 0)) != SAVE_VERSION:
		return {}
	return state


func save_state(state: Dictionary) -> Error:
	var config := ConfigFile.new()
	var load_error := config.load(SAVE_PATH)
	if load_error != OK and load_error != ERR_FILE_NOT_FOUND:
		push_warning("No se pudo leer el guardado anterior; se reemplazará.")
	config.set_value("progress", "state", state)
	return config.save(SAVE_PATH)


func clear_save() -> Error:
	if not FileAccess.file_exists(SAVE_PATH):
		return OK
	return DirAccess.remove_absolute(ProjectSettings.globalize_path(SAVE_PATH))
```

Cuatro decisiones que vale la pena defender:

1. `SAVE_VERSION`: si el archivo existe pero la versión no coincide, `load_state()` devuelve vacío en vez de cargar datos incompatibles. Prefiero que el juego arranque de cero antes que cargar basura y romperse.
2. `has_save()` no se conforma con que exista el archivo: también verifica que se pueda leer. Un archivo corrupto no debe mostrar "CONTINUAR".
3. `push_warning` en vez de `push_error` si el archivo anterior está roto: no es fatal, se reemplaza y se sigue.
4. `clear_save()` devuelve `OK` si el archivo no existe. Es idempotente: llamarlo dos veces no falla.

8.3 Cuándo se guarda

Guardo en tres momentos distintos, y cada uno cubre un riesgo:

Momento	Cómo	Qué riesgo cubre
Cada 20 segundos	`autosave_clock` en `_process()`	Cierre inesperado, batería, crash
Al salir de la escena	`_exit_tree()`	El jugador vuelve al menú
Al comprar algo	`_save_progress()` en cada compra	Gastar fichas y perder la mejora

world.gd · líneas 243-257 — guardado automático y al salir

```
func _exit_tree() -> void:
	if is_instance_valid(player):
		_save_progress()
	if is_instance_valid(radio_streamer):
		radio_streamer.call("stop")
	if is_instance_valid(radio_audio_player):
		radio_audio_player.stop()
		radio_audio_player.stream = null


func _process(delta: float) -> void:
	autosave_clock += delta
	if autosave_clock >= AUTOSAVE_SECONDS:
		autosave_clock = 0.0
		_save_progress()
```

El `_exit_tree()` además hace algo que no es guardar: **apaga la radio**. Si el jugador vuelve al menú con la radio sonando, la música seguiría y no habría forma de apagarla. Ese tipo de detalle es el que hace que un juego se sienta terminado o no.

8.4 Qué se guarda exactamente

world.gd · líneas 1500-1533 — el diccionario completo

```
func _save_progress() -> void:
	if not is_instance_valid(player) or not is_instance_valid(worker_sprite):
		return
	var state := {
		"version": 1,
		"economy_version": ECONOMY_VERSION,
		"field_layout_version": 2,
		"total_xp": total_xp,
		"coins": coins,
		"harvest_count": harvest_count,
		"golden_found": golden_found,
		"golden_crop_index": golden_crop_index,
		"golden_wheat_odds": golden_wheat_odds,
		"luck_upgrades": luck_upgrades,
		"worker_hired": worker_hired,
		"employee_count": employee_count,
		"worker_speed": worker_speed,
		"worker_speed_upgrades": worker_speed_upgrades,
		"extra_land_unlocked": extra_land_unlocked,
		"seed_inventory": seed_inventory.duplicate(),
		"selected_crop_type": selected_crop_type,
		"crop_types": crop_types.duplicate(),
		"crop_planted": crop_planted.duplicate(),
		"crop_growth_seconds": crop_growth_seconds.duplicate(),
		"new_zone_unlocked": new_zone_unlocked,
		"inside_new_zone": inside_new_zone,
		"weather_type": weather_type,
		"weather_timer": weather_timer,
		"player_position": player.global_position,
		"regrow_timers": regrow_timers.duplicate(),
	}
	var result := int(get_node("/root/GameSave").call("save_state", state))
	if result != OK:
		push_warning("No se pudo guardar el progreso de la granja.")
```

Dos cosas técnicas que importan mucho acá:

- `.duplicate()` **en los arrays.** Si guardo la referencia, el diccionario apunta al mismo array vivo y el guardado cambia mientras el juego sigue. Con `duplicate()` guardo una **foto** del estado en ese instante.
- **El guardado es una foto, no un delta.** Cada 20 segundos se escribe todo de nuevo. Es más simple que llevar cambios incrementales y, con 9 KB, no hay costo.

ERROR REAL #9 · EL GUARDADO INCOMPLETO

Mi primera versión guardaba fichas, XP y posición. Al volver a entrar, el campo estaba vacío y el clima siempre era soleado. El problema no era el archivo: era que **no había incluido esos datos en el diccionario**. Lo detecté escribiendo el bucle de juego en un papel y tachando qué variables cambiaban. Desde entonces, cada vez que agrego una variable de progreso, la agrego también acá y en `_restore_progress()`.

8.5 Restaurar: el orden también importa

Al cargar, primero se crean los cultivos y los sprites, y **después** se aplica el estado guardado. Si fuera al revés, no habría dónde escribir. Por eso `_restore_progress()` se llama al final del `_ready()`:

world.gd · líneas 218-241 — orden de inicialización

```
func _ready() -> void:
        randomize()
        var camera_zoom := float(get_node("/root/GameSettings").get("camera_zoom"))
        $Player/Camera2D.zoom = Vector2.ONE * camera_zoom
        _setup_terrain_layers()
        _setup_farm_art()
        _setup_crops()
        _setup_npcs()
        _setup_radio()
        _build_weather_visuals()
        _build_touch_controls()
        _build_battle_overlay()
        _build_roulette_overlay()
        _build_shop_overlay()
        _build_bulk_planting_overlay()
        _build_pause_overlay()
        _build_radio_overlay()
        player.interact_requested.connect(_on_player_interact)
        _restore_progress()          # <-- siempre al final
        _update_weather_visuals()
        _update_bulk_planting_ui()
        _update_status()
        _update_building_locks()
```

Esa lista es también una buena respuesta para “¿cómo se inicializa tu juego?”. Se ve el orden completo: primero el terreno, después el arte, después los objetos, después la interfaz, y recién al final los datos del jugador.

PARA DECIR EN EL ORAL

“El guardado es un diccionario con 25 claves que se escribe en un archivo de texto. Guardo cada 20 segundos, al salir de la escena y en cada compra. Y cuando cambié el diseño del campo, versioné el guardado para migrar los índices: un jugador que actualiza el juego no puede perder su granja.”

9 La radio: el problema más difícil

La radio fue, sin dudas, la parte que más me costó. No por la interfaz —un selector, un slider y tres botones— sino porque **reproducir audio en vivo dentro de un juego no es lo mismo que reproducir un archivo**. Este capítulo lo cuento completo, porque es el que mejor muestra cómo resolví un problema técnico real.



La radio abierta en el juego: selector de emisoras, estado (RADIO APAGADA), volumen, y los botones SINTONIZAR / APAGAR / VOLVER. La luz roja del aparato se pone verde cuando está al aire.

9.1 La idea

En la intro, el cuarto párrafo dice: *"Una vieja radio le traía voces y música de Argentina mientras sembraba"*. Eso no podía quedar solo en el texto. Quería que el jugador se acercara a un aparato en el campo, apretara `E` y **escuchara una radio de verdad**, con emisoras reales.

Y quería que fuera **ficción con marco real**: las emisoras existen, transmiten en vivo y se pueden verificar. Eso le da al juego una textura que ningún audio descargado puede dar.

9.2 El problema técnico

Un `AudioStreamPlayer` de Godot reproduce archivos o streams completos: cargás un `.ogg` o un `.mp3` y lo reproducís. Pero una radio por Internet es distinto:

- Es un **stream infinito**: nunca termina, no sabés la duración.
- Llega por **HTTP en trozos** (chunked) con un protocolo tipo Icecast/Shoutcast.
- Viene en **MP3** y a veces trae **metadatos** (el nombre del tema) mezclados en el flujo.
- Se puede **cortar** y hay que reconectar sin que el juego se congele.

Godot no trae eso de fábrica. La solución fue usar una **GDExtension**: una biblioteca nativa (C++) que se carga en el motor y expone una clase nueva a GDScript. En este caso, `IcyRadioStreamer`.

9.3 Qué es una GDExtension y qué aporta esta

Una GDExtension es código compilado que se agrega al motor sin recompilar Godot. Se distribuye como un archivo `.gdextension` que le dice al motor dónde están las bibliotecas para cada plataforma. En el proyecto hay bibliotecas para:

Plataforma	Archivo	¿La radio funciona?
Windows x86_64	`libicy_radio_streamer.windows.release.x86_64.dll`	Sí
Linux x86_64	`libicy_radio_streamer.linux.release.x86_64.so`	Sí
Android ARM64	`libicy_radio_streamer.android.release.arm64.so`	Sí
Web	— (no hay binario y la exportación web tiene `extensions_support=false`)	No: abre la emisora en una pestaña
iOS	— (el addon no trae binario iOS)	No: abre la emisora en Safari

Esa tabla no es un detalle: **es el diseño de la solución**. Cuando la extensión existe, el juego reproduce el stream. Cuando no existe, el juego abre la página web de la emisora (que tiene su propio reproductor). En los dos casos el jugador escucha radio; lo que cambia es dónde.

> QUÉ HACE LA EXTENSIÓN, EN UNA FRASE
>
> Descarga el stream en **un hilo aparte** (para no congelar el juego), lo decodifica a audio crudo, lo entrega a un `AudioStreamGenerator` dentro de un `AudioStreamPlayer` normal, y reconecta solo si se corta. El juego solo tiene que darle una URL y decirle `play()`.

9.4 Cómo la conecté (paso a paso)

La extensión funciona así: se crea un nodo `IcyRadioPlayer`, se le indica **qué** `AudioStreamPlayer` va a usar, y se conectan dos señales: `state_changed` y `stream_failed`.

world.gd · líneas 905-927 _setup_radio()

```
func _setup_radio() -> void:
	radio_audio_player = AudioStreamPlayer.new()
	radio_audio_player.name = "RadioAudioPlayer"
	radio_audio_player.process_mode = Node.PROCESS_MODE_ALWAYS
	radio_audio_player.volume_linear = float(get_node("/root/GameSettings").get("radio_volume"))
	add_child(radio_audio_player)

	if not ClassDB.class_exists("IcyRadioPlayer"):
		push_warning("No se encontró la extensión IcyRadioStreamer; la radio quedará
deshabilitada.")
		return

	radio_streamer = ClassDB.instantiate("IcyRadioPlayer") as Node
	if not is_instance_valid(radio_streamer):
		push_warning("No se pudo crear el reproductor de radio.")
		return
	radio_streamer.name = "IcyRadioPlayer"
	radio_streamer.process_mode = Node.PROCESS_MODE_ALWAYS
	radio_streamer.set("player_path", NodePath("../RadioAudioPlayer"))
	# Algunas emisoras Icecast mandan metadatos con UTF-8 inválido; los nombres los muestra nuestra UI.
	radio_streamer.set("parse_metadata", false)
	radio_streamer.connect("state_changed", _on_radio_state_changed)
	radio_streamer.connect("stream_failed", _on_radio_stream_failed)
	add_child(radio_streamer)
```

Las cuatro decisiones de este bloque, una por una:

1. `ClassDB.class_exists("IcyRadioPlayer")`: pregunto si la clase existe *antes* de usarla. Si la extensión no se cargó (por ejemplo en web o en iOS), el juego **no se rompe**: sigue andando sin radio nativa. Es una comprobación defensiva que me evitó un crash en la versión web.
2. `process_mode = PROCESS_MODE_ALWAYS`: el nodo de radio sigue procesando aunque el juego esté pausado. Así la música no se corta si el jugador abre el menú de pausa.
3. `player_path`: la extensión no reproduce por sí misma; escribe el audio en el `AudioStreamPlayer` que le indico. Eso permite usar el volumen del juego y que el audio pase por el mismo sistema de audio.
4. `parse_metadata = false`: descubrí que algunas emisoras mandan el nombre del tema con bytes inválidos y la extensión fallaba al parsearlo. Como yo muestro el nombre de la emisora en mi propia interfaz, no necesitaba los metadatos del stream. **Apagar una función que no usás es una solución válida.**

9.5 La máquina de estados

La radio no está “prendida o apagada”: pasa por varios estados mientras conecta. La extensión los informa y yo los traduzco a texto para el jugador:

world.gd · líneas 1937-1953 estados de la radio

```
func _on_radio_state_changed(state: int) -> void:
	_set_radio_led(state == 3)
	if not is_instance_valid(radio_status_label):
		return
	match state:
		0:
			radio_status_label.text = "RADIO APAGADA"
		1:
			radio_status_label.text = "BUSCANDO SEÑAL..."
		2:
			radio_status_label.text = "CARGANDO AUDIO..."
		3:
			radio_status_label.text = "AL AIRE · EN VIVO"
		4:
			radio_status_label.text = "REINTENTANDO..."
		_:
			radio_status_label.text = "SIN SEÑAL"
```

Estado	Qué significa	Qué ve el jugador
0	Detenido	RADIO APAGADA (luz roja)
1	Conectando al servidor	BUSCANDO SEÑAL...
2	Llenando el buffer de audio	CARGANDO AUDIO...
3	Reproduciendo	AL AIRE · EN VIVO (luz verde)
4	Se cortó y está reintentando	REINTENTANDO...

Esa luz del aparato no es decorativa: es la misma información que el texto, pero visible de reojo mientras caminás por el mapa. La dibujé como un `Polygon2D` de 2 × 2 píxeles que cambia de color:

world.gd · líneas 1965-1967 el LED de la radio

```
func _set_radio_led(is_on_air: bool) -> void:
	if is_instance_valid(radio_led):
		radio_led.color = Color("78d69b") if is_on_air else Color("8a3d35")
```

9.6 Sintonizar

world.gd · líneas 1899-1918 _play_selected_radio()

```
func _play_selected_radio() -> void:
	var station: Dictionary = RADIO_STATIONS[selected_radio_station]
	radio_now_playing_label.text = station["name"]
	if bool(station.get("external_only", false)) or not is_instance_valid(radio_streamer):
		if is_instance_valid(radio_streamer):
			_stop_radio()
		# En iOS, web y cualquier plataforma sin la extensión nativa, abrimos la página
		# web de la emisora (con su reproductor) en vez de la URL cruda del stream.
		var listen_url := str(station.get("homepage", station["url"]))
		OS.shell_open(listen_url)
		radio_status_label.text = "ABRIENDO LA EMISORA EN EL NAVEGADOR"
		return
	radio_status_label.text = "BUSCANDO SEÑAL..."
	var result := int(radio_streamer.call(
		"play",
		station["url"],
		PackedStringArray(["User-Agent: Aventura2D/1.0"])
	))
	if result != OK:
		radio_status_label.text = "NO SE PUDO INICIAR LA EMISORA"
```

Esta función tiene **dos caminos** y esa es toda la estrategia multiplataforma:

- **Camino nativo** (Windows, Android, Linux): le paso la URL del stream a la extensión y ella se encarga. El segundo parámetro es el **User-Agent**: algunas emisoras rechazan pedidos sin uno, como si fueran un navegador desconocido. Mandar `Aventura2D/1.0` fue la diferencia entre “403 Forbidden” y “al aire”.
- **Camino externo** (`external_only` o sin extensión): abro la **página web** de la emisora con `OS.shell_open()`. En Windows abre el navegador; en Android, Chrome; en iOS, Safari; en web, una pestaña nueva.

9.7 Las seis emisoras

world.gd · líneas 50-58 RADIO_STATIONS

```
# Public direct MP3 feeds from Argentine broadcasters (verified with the Radio Browser directory).
const RADIO_STATIONS := [
	{"name": "Radio Nacional AM 870", "url": "http://sa.mp3.icecast.magma.edge-
access.net:7200/sc_rad1", "homepage": "https://www.radionacional.com.ar/"},
	{"name": "Nacional Folklórica 98.7", "url": ".../sc_rad38", "homepage":
"https://www.radionacional.com.ar/nacional-folklorica-1/"},
	{"name": "Nacional Clásica 96.7", "url": ".../sc_rad37", "homepage":
"https://www.radionacional.com.ar/nacional-clasica/"},
	{"name": "Nacional Rock 93.7", "url": ".../sc_rad39", "homepage":
"https://www.radionacional.com.ar/nacional-rock/"},
	{"name": "Radio Argentina AM 570", "url": "http://server.laradio.online:25224/live.mp3",
"homepage": "https://radioarg.net/argentina-am-570"},
	{"name": "LU6 Atlántica · Mar del Plata (AAC+)", "url":
"https://streaming.sonando.online:8080/stream", "external_only": true, "homepage":
"https://lu6.com.ar/escuchanos-en-vivo/"},
]
```

Cada entrada tiene tres datos: nombre para mostrar, URL del stream y página de la emisora. La sexta, además, tiene `external_only: true`, y esa marca tiene una historia.

ERROR REAL #10 · LU6 SONABA EN SILENCIO

Agregué LU6 Mar del Plata porque me parecía una emisora emblemática. La extensión conectaba, el estado llegaba a 3 (al aire)... y no se escuchaba nada. El motivo: LU6 transmite en **AAC+**, y la extensión solo decodifica **MP3**. Conectaba, pero el decodificador no entendía el flujo.

Podría haberla borrado, pero es una radio real y quería conservarla. La solución fue marcarla como `external_only`: al elegirla, el juego abre su página web, donde el reproductor oficial sí soporta AAC. **No pude resolverlo dentro del juego, pero sí resolví la experiencia del jugador.**

CÓMO VERIFIQUÉ LAS URLS

No copié links de un blog. Usé el directorio público *Radio Browser* para obtener los flujos directos, y probé cada uno: que respondiera, que fuera MP3 y que el estado llegara a "AL AIRE". Las URLs de radio cambian, así que dejé la lista como constante: si una deja de funcionar, se reemplaza en una línea.

9.8 El aparato: la radio también es un objeto del mundo

La radio no está solo en la interfaz: es un mueble en el campo. La dibujé entera con polígonos, sin usar una imagen:

world.gd · líneas 930-985 (extracto) — el mueble de la radio

```
func _add_radio_device(at: Vector2) -> void:
        var radio := StaticBody2D.new()
        radio.name = "VillageRadio"
        radio.position = at
        radio.collision_layer = 1
        radio.collision_mask = 1
        radio.z_index = 1
        var cabinet := Polygon2D.new()
        cabinet.polygon = PackedVector2Array([Vector2(-15, -10), Vector2(15, -10), Vector2(15, 7),
Vector2(-15, 7)])
        cabinet.color = Color("9a683e")
        radio.add_child(cabinet)
        var face := Polygon2D.new()
        face.position = Vector2(0, -2)
        face.polygon = PackedVector2Array([Vector2(-12, -5), Vector2(12, -5), Vector2(12, 4), Vector2(-12,
4)])
        face.color = Color("42352b")
        radio.add_child(face)
        # ... display verde, rejilla, dos perillas, antena y colisión
        map_objects.add_child(radio)
        map_objects.add_child(_make_world_label("RADIO · E", at + Vector2(-17, -30), Color("fff0bf")))
```

Son siete piezas: sombra, gabinete marrón, frente oscuro, visor verde, tres barritas de rejilla, dos perillas y una antena (`Line2D`). Todo con coordenadas relativas al objeto, así puedo mover la radio cambiando `RADIO_POSITION` y nada más.

Y el cartel `"RADIO · E"` flotando arriba es importante: sin él, el jugador no sabría que ese mueble se puede usar. En un juego sin tutoriales, **los carteles son el tutorial**.

9.9 La interfaz de la radio

world.gd · líneas 1836-1869 — selector, volumen y botones

```
        radio_station_picker = OptionButton.new()
        radio_station_picker.position = Vector2(43, 58)
        radio_station_picker.size = Vector2(234, 21)
        radio_station_picker.add_theme_font_size_override("font_size", 8)
        for station in RADIO_STATIONS:
                radio_station_picker.add_item(station["name"])
        radio_station_picker.select(selected_radio_station)
        radio_station_picker.item_selected.connect(_on_radio_station_selected)
        radio_overlay.add_child(radio_station_picker)

        radio_volume_slider = HSlider.new()
        radio_volume_slider.min_value = 0.0
        radio_volume_slider.max_value = 1.0
        radio_volume_slider.step = 0.05
        radio_volume_slider.value = float(get_node("/root/GameSettings").get("radio_volume"))
        radio_volume_slider.value_changed.connect(_on_radio_volume_changed)

        var tune_button := _ui_button(radio_overlay, "SINTONIZAR", Vector2(42, 127), Vector2(87, 22))
        var stop_button := _ui_button(radio_overlay, "APAGAR", Vector2(135, 127), Vector2(62, 22))
        var close_button := _ui_button(radio_overlay, "VOLVER", Vector2(205, 127), Vector2(72, 22))
        tune_button.pressed.connect(_play_selected_radio)
        stop_button.pressed.connect(_stop_radio)
        close_button.pressed.connect(_close_radio)

        if not is_instance_valid(radio_streamer):
                radio_status_label.text = "SE ABRIRÁ EN EL NAVEGADOR"
                tune_button.text = "ESCUCHAR WEB"
```

Las dos últimas líneas son **adaptación de la interfaz a la capacidad real**: si no hay extensión, el botón no dice “SINTONIZAR” (que promete audio interno) sino “ESCUCHAR WEB”, y el estado avisa desde el principio qué va a pasar. Nada peor que un botón que promete algo que no hace.

El volumen, además, se guarda en `settings.cfg`: si el jugador lo deja al 30%, la próxima vez arranca al 30%. Y como la radio se puede cerrar sin apagarla, el mensaje lo dice: *“Volviste a la granja. La radio puede seguir sonando.”*

9.10 Qué aprendí con la radio

1. **Verificar capacidades antes de usar una API.** `ClassDB.class_exists()` convirtió un crash en un mensaje.
2. **Un problema de formato no siempre se arregla con código.** LU6 (AAC) se resolvió cambiando la estrategia, no el decodificador.
3. **El audio en vivo es asincrónico.** Todo pasa en otro hilo y en otro tiempo: por eso la interfaz muestra estados y no un simple “on/off”.
4. **Los servidores tienen reglas.** Mandar User-Agent, usar HTTPS donde se puede, y aceptar que algunas emisoras bloquean.
5. **La ficción necesita sustento real.** Que las emisoras existan hace que la radio del pueblo no se sienta un adorno.

PARA DECIR EN EL ORAL

"La radio fue el desafío más grande. Godot no reproduce streams Icecast de fábrica, así que usé una GDExtension que descarga y decodifica en un hilo aparte. El juego le pasa la URL de la emisora y ella escribe el audio en un AudioStreamPlayer. Como la extensión no existe para web ni para iOS, ahí el juego detecta que no está, cambia el botón a 'Escuchar web' y abre la página de la emisora. Es una sola función con dos caminos, y el jugador escucha radio en las cuatro plataformas."

"Y hay un caso que me gusta contar: una emisora transmite en AAC+, que la extensión no decodifica. En vez de borrarla, la marqué como 'solo externa': el juego abre su reproductor oficial. No resolví el formato, resolví la experiencia."

SI TE PREGUNTAN "¿Y POR QUÉ NO USASTE UN ARCHIVO DE AUDIO?"

Porque un archivo se repite y se nota. La radio en vivo cambia: hay música distinta, hablan, hay publicidades. Eso hace que el mundo se sienta conectado con algo real. Además, técnicamente fue el problema más interesante del proyecto: obligó a entender hilos, protocolos y diferencias entre plataformas.

10 Llevarlo a todos lados: exportar y publicar

Tener el juego andando en el editor no es tener un juego. Un juego existe cuando otra persona lo puede abrir sin instalar Godot. Este capítulo es el que convierte el proyecto en algo que se descarga: cuatro exportaciones, una página web y un código QR.

10.1 Los cuatro presets

Godot exporta desde *Proyecto → Exportar*. Cada plataforma es un **preset** guardado en `export_presets.cfg`, con su ruta de salida y sus opciones:

Preset	Plataforma	Sale a	Resultado
Android (ARM64)	Android	`Aventura2D.apk`	28,9 MB
Web	Web	`web/index.html`	Carpeta con HTML + WASM + PCK
iOS	iOS	`ios_build/`	Proyecto Xcode → .ipa
Windows Desktop	Windows	`pc/Aventura2D.exe`	ZIP de 37,5 MB

Un detalle que casi me cuesta un día: **las plantillas de exportación tienen que ser exactamente de la misma versión que el editor**. Si el editor es 4.7.2 y las plantillas son 4.6, la exportación falla con un error que no dice nada claro. En el proyecto está resuelto: la carpeta de plantillas es `4.7.2.stable` y coincide con el motor.

10.2 Windows: el ZIP que se descarga

La exportación de Windows genera **dos archivos que van juntos**: el ejecutable y el paquete de datos.

export_presets.cfg · preset Windows — exportación de escritorio

```
[preset.3]

name="Windows Desktop"
platform="Windows Desktop"
export_filter="all_resources"
export_path="pc/Aventura2D.exe"
```

El resultado en la carpeta `pc/` es `Aventura2D.exe` + `Aventura2D.pck` (y la DLL de la extensión de radio). **Si mandás solo el .exe, el juego no arranca**: el .exe es el motor y el .pck son los datos. Por eso lo publiqué como ZIP.

> **POR QUÉ ZIP Y NO INSTALADOR**
>
> Un instalador (NSIS, Inno Setup) agrega pasos y antivirus sospechando. Un ZIP se descomprime y se ejecuta: cero fricción para alguien que quiere probar el juego. Para un proyecto escolar, es la decisión correcta.

10.3 Web: el juego en un link

La versión web fue la que más me sirvió para mostrar el juego: no hay que instalar nada, se abre en el navegador. Pero tiene tres condiciones que hay que respetar:

1. **Necesita un servidor HTTP.** El `index.html` exportado **no funciona** haciendo doble clic (protocolo `file://`): el navegador bloquea la carga del `.wasm` por seguridad. Se prueba con un servidor local o se publica en la web.
2. **El renderer tiene que ser Compatibility.** Si el proyecto usara Forward+, la exportación web directamente no existe.
3. **No hay extensiones nativas.** El preset web tiene `variant/extensions_support=false`. Por eso la radio en web usa el camino externo (abre la emisora en otra pestaña).

```
export_presets.cfg · preset Web (extracto)                    opciones que importan

name="Web"
platform="Web"
export_path="web/index.html"
...
variant/extensions_support=false
variant/thread_support=false
html/canvas_resize_policy=2
html/focus_canvas_on_start=true
```

`thread_support = false` es importante: sin hilos, el juego es compatible con **cualquier** hosting estático. Si activara hilos, el servidor tendría que mandar cabeceras especiales (*Cross-Origin-Opener-Policy* y *Cross-Origin-Embedder-Policy*) y GitHub Pages no las permite. Preferí compatibilidad total antes que unos milisegundos de rendimiento.

10.4 Android: el APK

Android fue la plataforma más directa, con una excepción: la radio necesita Internet y los streams son **HTTP** (no HTTPS). En el preset están declarados los permisos y la arquitectura:

```
export_presets.cfg · preset Android                    arquitectura, versión y permisos

name="Android (ARM64)"
platform="Android"
export_path="Aventura2D.apk"
architectures/armeabi-v7a=false
architectures/arm64-v8a=true
version/code=1
version/name="1.0"
package/unique_name="com.aventura2d.granja"
permissions/internet=true
```

Cuatro datos para explicar:

- `arm64-v8a` **solamente:** el APK pesa 28,9 MB en vez de ~90 MB porque no incluye binarios para arquitecturas viejas. Todos los celulares de los últimos años son ARM64.
- `package/unique_name`**:** es el identificador único de la app (`com.aventura2d.granja`). Android no permite dos apps con el mismo ID: es como el DNI de la aplicación.
- `permissions/internet = true`**:** sin este permiso la radio no puede conectar. Es el único permiso que el juego pide.
- **APK de depuración firmado:** para instalar por fuera de la tienda alcanza. Google Play pediría un AAB y una firma propia.

ERROR REAL #11 · EL APK NO INSTALABA

Al principio generaba el APK sin firmar y Android lo rechazaba con “aplicación no instalada”. Un APK **siempre** tiene que estar firmado, aunque sea con una clave de depuración. Se resuelve en *Proyecto → Exportar → Android → Keystore*, generando un keystore de debug. Después de eso instaló a la primera.

10.5 iOS: el problema de no tener una Mac

Acá está la historia más larga del capítulo. Para compilar para iPhone hace falta **Xcode**, que solo existe en macOS. Yo no tengo una Mac. La solución fue usar **GitHub Actions**: los repositorios de GitHub incluyen máquinas virtuales con macOS gratis para proyectos públicos y con minutos limitados para privados.

El workflow `build-ios.yml` hace todo el proceso en la nube, en cinco pasos:

.github/workflows/build-ios.yml · líneas 1-32 — disparador y preparación

```yaml
name: Build iOS

on:
  push:
    branches: [main]
  workflow_dispatch:

jobs:
  build:
    runs-on: macos-latest
    timeout-minutes: 60

    steps:
      - name: Descargar el proyecto
        uses: actions/checkout@v4

      - name: Descargar Godot 4.7.2
        run: |
          curl -L --retry 3 -o godot.zip \
            https://github.com/godotengine/godot-builds/releases/download/4.7.2-stable/Godot_v4.7.2-
stable_macos.universal.zip
          unzip -q godot.zip
          chmod +x Godot.app/Contents/MacOS/Godot
          ./Godot.app/Contents/MacOS/Godot --version

      - name: Descargar plantillas de exportacion
        run: |
          curl -L --retry 3 -o templates.tpz \
            https://github.com/godotengine/godot-builds/releases/download/4.7.2-stable/Godot_v4.7.2-
stable_export_templates.tpz
          mkdir -p "$HOME/Library/Application Support/Godot/export_templates"
          unzip -q templates.tpz -d /tmp/tpl
          mv /tmp/tpl/templates "$HOME/Library/Application Support/Godot/export_templates/4.7.2.stable"
```

Los pasos siguientes son los interesantes:

build-ios.yml · líneas 34-83 exportar, compilar y empaquetar

```yaml
      - name: Exportar el proyecto Xcode
        # Godot intenta firmar y compilar el .ipa por su cuenta y falla
        # (el team del preset es de relleno). El proyecto Xcode queda generado.
        continue-on-error: true
        run: |
          mkdir -p build
          ./Godot.app/Contents/MacOS/Godot --headless --path . \
            --export-release iOS "$PWD/build/Aventura2D.ipa" || true
          ls -la build

      - name: Comprobar que el proyecto Xcode existe
        run: |
          if [ ! -d build/Aventura2D.xcodeproj ]; then
            echo "No se genero el proyecto Xcode."
            exit 1
          fi

      - name: Compilar sin firma
        run: |
          cd build
          xcodebuild \
            -project Aventura2D.xcodeproj \
            -scheme Aventura2D \
            -configuration Release \
            -sdk iphoneos \
            -destination 'generic/platform=iOS' \
            -derivedDataPath dd \
            CODE_SIGNING_ALLOWED=NO \
            CODE_SIGNING_REQUIRED=NO \
            CODE_SIGN_IDENTITY="" \
            build

      - name: Empaquetar el .ipa
        run: |
          mkdir -p out/Payload
          cp -R build/dd/Build/Products/Release-iphoneos/Aventura2D.app out/Payload/
          cd out
          zip -qry Aventura2D.ipa Payload

      - name: Subir el .ipa
        uses: actions/upload-artifact@v4
        with:
          name: Aventura2D-ipa
          path: out/Aventura2D.ipa
          if-no-files-found: error
          retention-days: 30
```

La clave está en **compilar sin firma**. Un `.ipa` es simplemente un ZIP con una carpeta `Payload` que contiene el `.app`. Godot intenta firmarlo él mismo y falla porque el *team ID* del preset es de relleno; por eso el paso tiene `continue-on-error: true` y lo que importa es que **genere el proyecto Xcode**. Después, `xcodebuild` compila sin firma y el ZIP final es un .ipa que se puede firmar después.

10.6 Instalarlo en el iPhone

Un .ipa sin firmar no se puede instalar: Apple exige una firma. Como no tengo cuenta de desarrollador paga, usé **Sideloadly**, una herramienta gratuita que firma con un certificado personal y lo instala por USB.

1. Se conecta el iPhone por cable y se abre Sideloadly.
2. Se arrastra el `.ipa` y se ingresa el Apple ID (la herramienta usa una contraseña de aplicación).
3. Sideloadly firma el paquete y lo instala en el teléfono.

El certificado gratuito **dura 7 días**. Pasado ese plazo, la app deja de abrir hasta volver a firmarla con Sideloadly. Es la limitación real de instalar sin cuenta de desarrollador, y es importante saberla para no prometer algo que no es: **la versión de iPhone es para probar, no para distribuir**.

ACTUALIZAR SIN PERDER EL GUARDADO

Instalar la versión nueva **encima** de la anterior (mismo bundle ID) conserva los datos, porque el guardado vive en la carpeta de la app. Eso me permitió arreglar la radio y actualizar el juego sin que el jugador perdiera la granja. Si se desinstala primero, se borra todo.

10.7 Publicar: GitHub Pages y Releases

Para que el juego se pueda jugar y descargar sin que yo esté prendido, usé dos funciones de GitHub:

Dónde	Qué publica	Para qué sirve
GitHub Pages	La exportación web (HTML + WASM + PCK)	Jugar online desde un link, sin descargar
Releases	El ZIP de Windows y el APK de Android	Descargar la versión de escritorio o instalar en el celular
Página de descargas	Un HTML propio con los tres botones	Un solo lugar donde el jugador elige qué bajar

Los links quedaron así:

Versión	Enlace
Jugar online	`garciarenaalejandro453-png.github.io/aventura2d-web/`
Descargas	`garciarenaalejandro453-png.github.io/aventura2d-web/descargas.html`
PC (ZIP)	`github.com/…/releases/download/v1.0/Aventura2D-PC-Windows.zip`
Android (APK)	`github.com/…/releases/download/v1.0/Aventura2D-Android.apk`

ERROR REAL #12 · EL JUEGO WEB ANDABA EN MI PC Y NO EN EL SERVIDOR

Después de subir la exportación web, el juego quedaba en negro. El motivo: el `.pck` se había subido con un nombre distinto al que esperaba el HTML (mayúsculas). En Linux (que es lo que usa GitHub Pages) los nombres de archivo distinguen mayúsculas y minúsculas; en Windows no. Lo resolví subiendo la carpeta de exportación completa, sin renombrar nada a mano.

10.8 El QR: del papel al teléfono

Para la entrega impresa, un link escrito no sirve: nadie lo tipea. Generé un **código QR** que apunta a la página de descargas, así el profesor lo escanea con el celular y llega al juego en dos toques.

El QR no lo dibujé yo: lo generé con una herramienta a partir de la URL exacta y después lo **verifiqué** decodificándolo con un lector. Ese paso es importante: un QR mal generado se ve perfecto y no lleva a ningún lado. En el proyecto está el QR en PNG y en PDF, listo para imprimir.

POR QUÉ LA PÁGINA DE DESCARGAS Y NO EL JUEGO DIRECTO

Porque el QR apunta a un lugar que **no cambia**: la página de descargas. Si mañana actualizo los archivos, el QR sigue sirviendo. Si hubiera apuntado al ZIP, cada versión nueva necesitaría un QR nuevo. Es una decisión de diseño pensando en el mantenimiento.

PARA DECIR EN EL ORAL

"El mismo proyecto exporta a cuatro plataformas: Windows en un ZIP, Android en un APK, web en GitHub Pages e iOS en un .ipa. Como no tengo Mac, la compilación de iOS la hace GitHub Actions en una máquina macOS: descarga Godot, exporta el proyecto Xcode, compila sin firma y devuelve el .ipa. Después lo firmo con Sideloadly para instalarlo en el teléfono."

"Y todo llega por un QR que apunta a una página de descargas fija, así el código impreso nunca queda viejo."

11 Errores que me encontré y cómo los resolví

Esta es la parte que más sirve en una defensa. Cualquiera puede mostrar un juego que funciona; lo que muestra criterio es contar **qué se rompió y cómo lo pensaste**. Estos son los doce errores reales del proyecto, resumidos para repasar.

#	Error	Síntoma	Solución
1	Querer abarcar todo	Proyecto enorme, nada terminado	Recortar a un bucle jugable y terminarlo
2	Color de fondo	Borde gris donde faltaba terreno	`default_clear_color` celeste
3	Diagonal rápida	El personaje corría en diagonal	`direction.normalized()`
4	Zoom no aplicado	El ajuste no se veía en el juego	Aplicar zoom en el `_ready()` del mundo
5	Escena en negro	La transición no descubría	`Tween` en el autoload en vez de `await`
6	Cultivos invisibles	Surcos plantados sin planta al cargar	Restaurar `visible` y sprites
7	Empleado invisible	Dos empleados en la misma posición	Desplazarlos en grilla
8	Gaviota trabada	Vibraba contra los árboles	Sacarle colisión con el escenario
9	Guardado incompleto	Se perdían cultivos y clima	Guardar las 25 claves del estado
10	LU6 en silencio	Estado “al aire” sin audio	`external_only` → abrir la web
11	APK no instalaba	“Aplicación no instalada”	Firmar con keystore de debug
12	Web en negro	Andaba local y no en Pages	Subir la carpeta sin renombrar (mayúsculas)

11.1 Los tres que más me enseñaron

El guardado incompleto (#9)

Me enseñó que **el estado del juego hay que diseñarlo antes**, no después. La solución no fue “agregar las variables que faltaban”, fue entender que el guardado es una **foto del estado completo** y que cualquier variable nueva de progreso tiene que entrar ahí y en la restauración. Desde entonces tengo una regla: si agrego progreso, toco dos funciones (`_save_progress` y `_restore_progress`) en el mismo commit.

La radio en silencio (#10)

Me enseñó que **no todos los problemas se resuelven con código**. El decodificador no soporta AAC y no iba a escribir un decodificador en C++ para un trabajo escolar. La solución correcta fue **cambiar la estrategia**: abrir el reproductor oficial de esa emisora. El jugador escucha lo mismo; el juego no miente.

El juego web en negro (#12)

Me enseñó que **el entorno de prueba no es el entorno real**. En Windows los nombres de archivo no distinguen mayúsculas; en el servidor Linux sí. Desde entonces, cuando algo funciona en mi máquina y no en el servidor, lo primero que reviso son las rutas y los nombres exactos.

CÓMO PROBABA DESPUÉS DE CADA CAMBIO

Tenía una lista corta de **prueba de humo**: abrir el juego, entrar a la partida, plantar, esperar a que crezca, cosechar, abrir la tienda, sintonizar la radio, guardar, cerrar y volver a abrir. Si los ocho pasos funcionaban, el cambio no había roto lo importante. Es una prueba manual, pero sistemática: la misma siempre.

12 Guion para el oral y preguntas de defensa

Este capítulo está pensado para el día de la presentación. Primero va un **guion de cinco minutos** con marcas de tiempo, después **treinta preguntas con respuesta** y al final algunos consejos prácticos.

12.1 Guion de 5 minutos

MINUTO 0:00 — PRESENTACIÓN

"Mi proyecto es Aventura 2D, un juego de granja en pixel art hecho con Godot 4.7. Se puede jugar en Windows, en Android, en el navegador y en iPhone. La idea es simple: llegás a una granja heredada, sembrás, cosechás, vendés y mejorás. Y mientras jugás, suena una radio argentina de verdad."

MINUTO 0:30 — EL BUCLE DE JUEGO

"El corazón del juego es el ciclo plantar-esperar-cosechar-mejorar. Hay tres cultivos con tiempos distintos: zanahoria 60 segundos, tomate 75 y trigo 90. El clima no es decorativo: multiplica la velocidad de crecimiento. Con lluvia crecen un 60% más rápido; con sequía, un 40% más lento. Eso hace que el jugador mire el clima antes de decidir qué plantar."

MINUTO 1:15 — LA TÉCNICA

"El juego está en GDScript, con resolución interna de 320 por 180 escalada por múltiplos enteros para que el pixel art no se deforme. Uso tres autoloads: uno para ajustes, uno para el guardado y uno para las transiciones. La interfaz está construida por código, con una sola función que define el estilo de todos los botones."

MINUTO 2:00 — LA RADIO (EL PLATO FUERTE)

"La parte más difícil fue la radio. Godot no reproduce streams Icecast de fábrica: un stream es infinito, llega en trozos y puede cortarse. Usé una GDExtension que descarga y decodifica en un hilo aparte y escribe el audio en un AudioStreamPlayer. El juego le pasa la URL de la emisora y ella informa el estado: buscando señal, cargando, al aire, reintentando.

"Como la extensión no existe para web ni para iOS, ahí el juego detecta que no está, cambia el botón a 'Escuchar web' y abre la página de la emisora. Y hay un caso especial: una emisora transmite en AAC+, que la extensión no decodifica; la marqué como 'solo externa' y abre su reproductor oficial. No resolví el formato, resolví la experiencia."

MINUTO 3:15 — EL GUARDADO

"El guardado es un archivo de texto con veinticinco claves: nivel, XP, fichas, estado de cada surco, clima, empleados y posición del jugador. Guardo cada veinte segundos, al salir y en cada compra. Cuando cambié el diseño del campo, versioné el guardado para migrar los índices: un jugador que actualiza no pierde su granja."

MINUTO 3:55 — LA PUBLICACIÓN

"El mismo proyecto exporta a cuatro plataformas. Windows en un ZIP, Android en un APK, web en GitHub Pages, e iOS en un .ipa. Como no tengo Mac, la compilación de iOS la hace GitHub Actions en una máquina macOS: descarga Godot, exporta el proyecto Xcode, compila sin firma y devuelve el .ipa. Después lo firmo con Sideloadly, que dura siete días porque es certificado gratuito.

"Todo llega por un código QR que apunta a una página de descargas fija."

MINUTO 4:35 — CIERRE

"Si tuviera que resumir lo que aprendí: un juego no es una idea, es un montón de decisiones chicas. La más importante fue recortar: tenía una lista enorme de ideas y me quedé con lo que podía terminar y hacer funcionar. Preferí un juego chico que se puede jugar en cuatro plataformas antes que uno enorme a medias."

12.2 Treinta preguntas con respuesta

Sobre el juego

1. ¿De qué se trata el juego?

Mateo hereda una granja, la reconstruye sembrando y cosechando, y hace crecer el pueblo. El progreso va de una huerta chica a nueve campos grandes.

2. ¿Cuál es el objetivo del jugador?

No hay un final: el objetivo es progresar. Subir de nivel desbloquea el granero (nivel 5), la ruleta (nivel 10) y el sector nuevo (nivel 50). El "final" es tener la granja completa.

3. ¿Por qué una granja y no otro género?

Porque el progreso es visible y constante, y porque el ciclo plantar-cosechar se puede terminar bien con los recursos de un proyecto escolar. Además me permitía justificar la radio en la historia.

4. ¿Cuánto dura una partida?

No tiene duración fija: los cultivos se regeneran. Una sesión normal va de diez minutos a una hora. Llegar al nivel 50 con un solo cultivo llevaría muchas horas; con el equipo de empleados, bastante menos.

5. ¿Qué se puede hacer además de plantar?

Comprar mejoras, contratar hasta tres empleados, pelear en el granero de duelos, apostar en la ruleta, hablar con el vecino del pueblo y escuchar seis radios argentinas.

Sobre la programación

6. ¿Por qué Godot y no Unity?

Porque es gratis, liviano, exporta a las cuatro plataformas desde el mismo proyecto y GDScript se parece a Python. Unity con 2D pixel art era más peso y más configuración para el mismo resultado.

7. ¿Qué es un nodo en Godot?

La unidad mínima del motor: un sprite, una cámara, un temporizador, un cuerpo con física. Cada nodo hace una cosa y se combinan en un árbol llamado escena.

8. ¿Qué es un autoload y cuántos usás?

Un script que se carga una sola vez al arrancar y está disponible en todo el juego. Uso tres: GameSettings, GameSave y SceneTransition.

9. ¿Por qué la resolución es 320 × 180?

Porque es 16:9 y al escalarla por múltiplos enteros cada píxel mide exactamente lo mismo. Evita el pixel art deformado y hace que corra en cualquier celular.

10. ¿Por qué el personaje usa CharacterBody2D y no RigidBody2D?

Porque no quiero física realista: quiero que se mueva exactamente a la velocidad que digo y que se frene al soltar la tecla. RigidBody2D agregaría inercia y rebotes que no sirven en un juego de granja.

11. ¿Qué hace `normalized()` y por qué es importante?

Deja el vector de movimiento con longitud 1. Sin eso, la diagonal sería 1,41 veces más rápida, porque la diagonal de un cuadrado mide raíz de 2. Es un error clásico y muy visible.

12. ¿Por qué redondeás la posición del jugador?

Porque en pixel art una posición con decimales hace que el sprite vibre o se vea borroso. Redondear a píxeles enteros mantiene el arte nítido.

13. ¿Cómo se comunican el jugador y el mundo?

Con una señal: el jugador emite `interact_requested` con su posición y el mundo decide si eso es una radio, una tienda o un cultivo. Así puedo agregar objetos interactivos sin tocar el script del jugador.

14. ¿Por qué construiste la interfaz por código?

Por consistencia (una sola función de estilo para todos los botones), por menos archivos (tres vistas en una escena) y porque a 320 × 180 posicionar a mano en el editor era más lento que escribirlo.

15. ¿Cómo funciona la transición entre escenas?

Es un autoload con un ColorRect y un shader que disuelve la pantalla en bloques. Una animación de cuatro pasos: cubrir, cambiar la escena, esperar 0,12 s y descubrir. Es autoload porque si perteneciera a la escena desaparecería en medio del cambio.

Sobre la radio

16. ¿Cómo funciona la radio?

Con una GDExtension llamada IcyRadioStreamer. Descarga el stream Icecast en un hilo aparte, lo decodifica a MP3 y escribe el audio en un AudioStreamPlayer. El juego solo le pasa la URL y recibe los cambios de estado.

17. ¿Por qué no usaste un AudioStreamPlayer normal?

Porque un AudioStreamPlayer reproduce archivos completos, y una radio es un stream infinito que llega en trozos, puede traer metadatos mezclados y se puede cortar. Godot no trae ese protocolo de fábrica.

18. ¿Qué es una GDExtension?

Una biblioteca nativa, compilada en C++, que se carga en el motor y expone clases nuevas a GDScript. No hay que recompilar Godot: se agrega como un addon con sus binarios por plataforma.

19. ¿Qué pasa en la versión web y en iPhone?

La extensión no tiene binario para esas plataformas, así que el juego lo detecta con `ClassDB.class_exists()`, cambia el botón a “Escuchar web” y abre la página de la emisora en una pestaña o en Safari.

20. ¿Cómo mostrás el estado de la radio?

Con una máquina de estados de cinco valores: apagada, buscando señal, cargando audio, al aire y reintentando. Se muestra en texto y también en el LED del aparato: rojo apagado, verde al aire.

21. ¿Qué fue lo más difícil de la radio?

Que todo pasa en otro hilo y en otro tiempo: no hay una respuesta inmediata. Por eso la interfaz muestra estados en vez de un on/off. Y el caso de la emisora en AAC+, que conectaba pero no sonaba.

22. ¿Por qué apagaste `parse_metadata`?

Porque algunas emisoras mandan el nombre del tema con bytes UTF-8 inválidos y el parser fallaba. Como yo muestro el nombre de la emisora en mi interfaz, no necesitaba esos metadatos.

23. ¿Las emisoras son reales?

Sí. Saqué los flujos directos del directorio público Radio Browser y probé cada uno hasta que llegara al estado "al aire". Son seis emisoras argentinas reales.

Sobre el guardado y la publicación

24. ¿Dónde se guarda la partida?

En la carpeta `user://` del proyecto, que en Windows es `%APPDATA%\Godot\app_userdata\Aventura 2D\`. Son dos archivos: `savegame.cfg` (progreso) y `settings.cfg` (preferencias).

25. ¿Cuándo se guarda?

Cada veinte segundos, al salir de la escena y en cada compra. Los tres momentos cubren cierres inesperados, salidas normales y compras.

26. ¿Qué pasa si el guardado es de una versión vieja?

Hay dos versiones: la del formato (`version`) y la de la economía (`economy_version`). Si la versión no coincide, no se carga basura; si es una versión anterior, se migran los datos, como los índices de los surcos o los niveles viejos.

27. ¿Cómo exportaste a iPhone sin una Mac?

Con GitHub Actions: una máquina macOS en la nube descarga Godot, exporta el proyecto Xcode, compila sin firma con xcodebuild y empaqueta el .ipa. Después lo firmo con Sideloadly para instalarlo por USB.

28. ¿Por qué la app de iPhone dura solo siete días?

Porque el certificado gratuito de Apple caduca a los siete días. Para distribuir de verdad haría falta una cuenta de desarrollador paga. La versión iOS es para probar, no para publicar.

29. ¿Qué diferencia hay entre GitHub Pages y Releases?

Pages publica la versión web, que se juega online desde un link. Releases guarda los archivos descargables: el ZIP de Windows y el APK de Android.

30. Si tuvieras que seguir, ¿qué agregarías?

Tres cosas: inercia suave en el movimiento, más tipos de cultivo con estaciones, y misiones de los vecinos. Nada de eso cambia la estructura: el bucle y el guardado ya están preparados para recibirlo.

12.3 Consejos para el día de la defensa

- **Empezá por el juego, no por el código.** Mostrá treinta segundos de juego y después explicá cómo está hecho. Si arrancás con el código, perdés a la mitad del público.

- **Tené el juego abierto y en la partida.** Nada de esperar la intro de 60 segundos en vivo. Y tené el QR a mano para que lo escaneen.
- **Cuando no sepas algo, decilo.** “Eso no lo resolví” seguido de “lo que sí hice fue…” vale muchísimo más que inventar. El caso de LU6 es el mejor ejemplo: no resolví el formato, resolví la experiencia.
- **Usá los números.** 320 × 180, 25 claves de guardado, 6 emisoras, 3 empleados, 2.000.000.000 de probabilidad. Los números concretos dan seguridad.
- **Practicá el minuto 2 (la radio).** Es la parte que más impresiona y la que más fácil se traba. Si esa parte sale fluida, el resto se acomoda.

LA FRASE PARA CERRAR

“Lo que más aprendí no fue a programar un juego: fue a decidir qué dejar afuera. Este juego tiene todo lo que se ve, y nada de lo que no pude terminar.”

A Anexo A. Glosario técnico

Autoload — Script que Godot carga una vez al arrancar y deja disponible en todo el juego.

APK — Paquete instalable de Android.

ATS — *App Transport Security*: regla de Apple que bloquea conexiones HTTP sin cifrar.

CharacterBody2D — Nodo de Godot para cuerpos que se mueven por código y colisionan.

Chunked — Forma de enviar datos por HTTP en trozos, sin saber el tamaño total.

ConfigFile — Formato de archivo de Godot con secciones y claves, en texto.

delta — Tiempo transcurrido desde el cuadro anterior, en segundos.

GDExtension — Biblioteca nativa que agrega clases al motor sin recompilarlo.

GDScript — Lenguaje de programación propio de Godot, parecido a Python.

HUD — Interfaz que se dibuja encima del juego (nivel, fichas, mensajes).

Icecast — Servidor de streaming de audio, el estándar de las radios por Internet.

Instanciar — Crear un objeto a partir de una escena o clase.

IPA — Paquete de aplicación de iOS (un ZIP con una carpeta Payload).

Keystore — Archivo con la clave con la que se firma un APK.

Normalizar — Dejar un vector con longitud 1, para que la dirección no altere la velocidad.

Pixel art — Estilo gráfico hecho píxel por píxel.

PCK — Paquete de datos del juego exportado por Godot.

Preset — Configuración de exportación para una plataforma.

Señal — Mecanismo de Godot para avisar que algo pasó, sin acoplar emisor y receptor.

Shader — Programa que se ejecuta en la placa de video para dibujar efectos.

Sprite — Imagen 2D que representa un objeto.

Stream — Flujo continuo de audio o video por red, sin duración definida.

Tween — Animación de valores en el tiempo (de 0 a 1 en 0,42 s).

WASM — Formato binario que ejecuta el navegador; es el "motor" del juego web.

Viewport — Área de dibujo del juego, acá de 320 × 180.

B Anexo B. Chuleta de una página

Números para recordar	Valor
Resolución interna / ventana	320 × 180 / 960 × 540
Mundo	5.120 × 5.120 píxeles
Velocidad del jugador	92 px/s
Cultivos	Trigo 90 s · Zanahoria 60 s · Tomate 75 s
Recompensas	10 / 15 / 15 fichas · 5 / 10 / 15 XP
XP por nivel (1-50)	500 XP
Autoguardado	Cada 20 segundos
Claves del guardado	25
Clima	4 estados, 65-105 s, multiplicadores 1 / 1,6 / 0,6 / 0,85
Empleados	3 (Mora, Tomi, Rosa), costos 500 / 900 / 1.500
Radios	6 emisoras (5 por streaming + LU6 externa)
Estados de la radio	0 apagada · 1 buscando · 2 cargando · 3 al aire · 4 reintentando
Desbloqueos	Nivel 5 granero · Nivel 10 ruleta · Nivel 50 sector nuevo
Scripts propios	7
Líneas de world.gd	+1.900

Las cinco frases que resumen el proyecto

1. “Un juego es una idea más cuatrocientas decisiones chicas.”
2. “El jugador solo se mueve y avisa que quiere interactuar; el mundo decide qué es.”
3. “El clima no decora: multiplica el crecimiento.”
4. “La radio funciona nativa donde hay extensión y en el navegador donde no.”
5. “El guardado es una foto del estado completo, versionada.”

Estructura para dibujar en el pizarrón

arquitectura del proyecto — de un vistazo

```
start_menu.tscn ──► intro.tscn ──► main.tscn (world.gd)
      │                  │              │
      └── GameSettings ──┴── GameSave ──┴── SceneTransition
                                              (autoloads)

world.gd
 ├── _setup_crops()      → arrays paralelos de surcos
 ├── _setup_radio()      → IcyRadioPlayer (GDExtension)
 ├── _build_weather_visuals() → tinte + partículas
 ├── _on_player_interact()    → distancias (radio 28, tienda 25…)
 ├── _process()          → crecimiento × clima + autoguardado
 └── _save_progress()    → 25 claves → savegame.cfg
```

C Anexo C. Código completo de referencia

Los dos scripts más cortos y más importantes, completos, para tener a mano si te piden leer código en la defensa.

C.1 player.gd (61 líneas)

scripts/player.gd — completo

```
extends CharacterBody2D

signal interact_requested(at_position: Vector2)

const SPEED := 92.0
const WORLD_SIZE := 5120.0
const EDGE_MARGIN := 20.0

@onready var sprite: Sprite2D = $Sprite2D

var facing_row := 0
var animation_clock := 0.0
var touch_directions := {"left": false, "right": false, "up": false, "down": false}


func _physics_process(delta: float) -> void:
	var horizontal := Input.get_axis("ui_left", "ui_right")
	var vertical := Input.get_axis("ui_up", "ui_down")
	if Input.is_key_pressed(KEY_A):
		horizontal -= 1.0
	if Input.is_key_pressed(KEY_D):
		horizontal += 1.0
	if Input.is_key_pressed(KEY_W):
		vertical -= 1.0
	if Input.is_key_pressed(KEY_S):
		vertical += 1.0
	horizontal += int(bool(touch_directions["right"])) - int(bool(touch_directions["left"]))
	vertical += int(bool(touch_directions["down"])) - int(bool(touch_directions["up"]))
	var direction := Vector2(horizontal, vertical)

	direction = direction.normalized()
	velocity = direction * SPEED
	move_and_slide()
	position = position.round()
	position.x = clampf(position.x, EDGE_MARGIN, WORLD_SIZE - EDGE_MARGIN)
	position.y = clampf(position.y, EDGE_MARGIN, WORLD_SIZE - EDGE_MARGIN)

	if direction != Vector2.ZERO:
		if absf(direction.x) > absf(direction.y):
			facing_row = 1 if direction.x < 0.0 else 2
		else:
			facing_row = 3 if direction.y < 0.0 else 0
		animation_clock += delta
		sprite.frame_coords = Vector2i(1 + int(animation_clock / 0.16) % 2, facing_row)
	else:
		animation_clock = 0.0
		sprite.frame_coords = Vector2i(0, facing_row)


func _input(event: InputEvent) -> void:
	if event is InputEventKey and event.pressed and not event.echo and event.keycode == KEY_E:
		request_interaction()


func set_touch_direction(direction_name: String, pressed: bool) -> void:
	if touch_directions.has(direction_name):
		touch_directions[direction_name] = pressed


func request_interaction() -> void:
	interact_requested.emit(global_position)
```

C.2 game_save.gd (33 líneas)

scripts/game_save.gd — completo

```
extends Node

const SAVE_PATH := "user://savegame.cfg"
const SAVE_VERSION := 1


func has_save() -> bool:
	return FileAccess.file_exists(SAVE_PATH) and not load_state().is_empty()


func load_state() -> Dictionary:
	var config := ConfigFile.new()
	if config.load(SAVE_PATH) != OK:
		return {}
	var state: Variant = config.get_value("progress", "state", {})
	if not state is Dictionary or int(state.get("version", 0)) != SAVE_VERSION:
		return {}
	return state


func save_state(state: Dictionary) -> Error:
	var config := ConfigFile.new()
	var load_error := config.load(SAVE_PATH)
	if load_error != OK and load_error != ERR_FILE_NOT_FOUND:
		push_warning("No se pudo leer el guardado anterior; se reemplazará.")
	config.set_value("progress", "state", state)
	return config.save(SAVE_PATH)


func clear_save() -> Error:
	if not FileAccess.file_exists(SAVE_PATH):
		return OK
	return DirAccess.remove_absolute(ProjectSettings.globalize_path(SAVE_PATH))
```

C.3 Mapa de archivos del proyecto

Archivo	Líneas	Responsabilidad
`scripts/world.gd`	+1.900	Todo el juego: mapa, cultivos, clima, tienda, radio, guardado
`scripts/start_menu.gd`	278	Menú principal, ajustes y confirmación
`scripts/intro.gd`	69	Cinco páginas de historia con máquina de escribir
`scripts/player.gd`	61	Movimiento, animación e interacción
`scripts/scene_transition.gd`	55	Cambio de escena con shader de píxeles
`scripts/game_settings.gd`	44	Zoom y volumen persistentes
`scripts/game_save.gd`	33	Leer, escribir y borrar el progreso
`.github/workflows/build-ios.yml`	83	Compilación de iOS en GitHub Actions

> *“Este cuaderno termina acá, pero el juego no: cada vez que alguien cosecha trigo y escucha la radio del pueblo, todo esto sigue pasando.”*

— Fin del cuaderno · Aventura 2D